

Politecnico di Torino

Master's Degree in Mechatronics Engineering

Academic Year 2025/2026

Graduation Session December 2026

Visual Autonomous Navigation for Horticulture

Supervisors:
Name Surname
Name Surname

Candidate:
Emanuele Pruiti

Abstract

Autonomous navigation for mobile agricultural robots in horticultural environments faces severe operational challenges under Global Navigation Satellite Systems (GNSS). Thick plant canopies, multi-path reflections, and greenhouse structures degrade or eliminate satellite reception, while Real-Time Kinematic (RTK) corrections incur high infrastructure costs. High-resolution 3D LiDAR sensors offer direct geometric measurements, but unit costs remain prohibitive for scalable commercial fleets, and spatial point clouds lack the chromatic and textural cues required to distinguish drivable soil from dense crop foliage.

This thesis presents a real-time, deterministic vision-based navigation system for row-following horticulture based on monocular RGB imagery. The architecture decouples offline model-assisted dataset curation from low-latency onboard execution. To minimize manual annotation overhead, the Segment Anything Model (SAM 3) generates initial binary corridor segmentations from prompt-guided field captures. For real-time inference on an embedded NVIDIA Jetson Orin Nano platform, a lightweight MobileNetV3 + LR-ASPP semantic segmentation network is deployed via ONNX Runtime with FP16 precision, executing at 20 Hz within a 15 ms inference budget.

From the predicted drivable corridor mask, navigation setpoints are extracted using a decoupled dual-edge geometric tracking algorithm. The region of interest is partitioned into horizontal scanline strips from bottom to top. Left and right physical crop boundaries are tracked independently using spatial continuity gates and fitted using Iteratively Reweighted Least Squares (IRLS) with a Cauchy M-estimator. The central navigation line is computed analytically from the converged boundary parameters.

The proposed algorithm is benchmarked against three established agricultural vision baselines: a global column-projection extractor (SegMin column histogram), a strip-centroid regression line extractor, and a contour-point line extractor fitted via Random Sample Consensus (RANSAC). Rather than claiming unconditional superiority, the thesis evaluates the structural trade-offs of each paradigm. The proposed dual-edge extractor eliminates the stochastic frame-to-frame angular jitter characteristic of RANSAC ($\sigma_\omega = 0.00^\circ$ versus non-zero variance) and prevents

unilateral leaf intrusions from biasing the corridor axis, a primary failure mode of centroid-based fitting. Conversely, experimental failure analysis formalizes key structural limitations of the proposed method: vulnerability to near-field seed occlusion, sensitivity of image-space jump gates to abrupt vehicle pitch and roll, inability of the straight-line model to track curved crop geometries or headland turnaround maneuvers, and boundary loss during extended unilateral crop gaps.

The perception and geometric extraction stages are validated offline across recorded horticultural datasets and benchmarked on the embedded NVIDIA Jetson Orin Nano, confirming real-time execution at 20 Hz with sub-15 ms latency without requiring secondary neural networks, stereo disparity maps, or LiDAR sensors. Finally, the thesis establishes the integrated control architecture and closed-loop field evaluation protocol on the Demetra four-wheel-steering (4WS) agricultural rover, using an auxiliary laser sensor to monitor physical corridor centering.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VIII

1 Introduction 1
- 1.1 Context and motivation . . . 1
- 1.2 Objectives of the thesis . . . 2
- 1.3 Organization of the work . . . 3

2 Background and Problem Formulation 4
- 2.1 Agricultural autonomous navigation . . . 4
  - 2.1.1 Navigation tasks and requirements . . . 4
  - 2.1.2 Environmental challenges . . . 4
- 2.2 Vision-based local guidance . . . 5
  - 2.2.1 Sensing modalities . . . 5
  - 2.2.2 Image-space reference extraction . . . 5
- 2.3 The segmentation-to-control pipeline . . . 7
  - 2.3.1 Perception stage . . . 7
  - 2.3.2 Geometric extraction stage . . . 7
  - 2.3.3 Control integration . . . 8

3 State of the Art 9
- 3.1 Classical image-processing methods . . . 9
- 3.2 Learning-based segmentation for agricultural navigation . . . 9
  - 3.2.1 Semantic segmentation approaches . . . 9
  - 3.2.2 Training data and generalization . . . 10
- 3.3 Semantic segmentation architectures . . . 10
  - 3.3.1 FCN, DeepLabV3, and mobile architectures . . . 10
  - 3.3.2 Transfer learning and pretraining . . . 10
- 3.4 Dataset preparation and model-assisted annotation . . . 11
  - 3.4.1 Manual annotation challenges . . . 11
  - 3.4.2 SAM and foundation models for segmentation . . . 11
- 3.5 Geometric extraction methods . . . 12

3.5.1 Line fitting and crop-edge detection . . . 12
3.5.2 Column-wise histogram methods . . . 12
3.5.3 Horizontal strips and adaptive multi-ROI methods . . . 12
3.5.4 Triangle Scan Method . . . 13
3.5.5 Bilateral boundary tracking versus centroid reduction . . . 13
3.5.6 Comparison of geometric representations . . . 15
3.6 System integration and experimental validation . . . 16
3.6.1 Perception and control integration . . . 16
3.6.2 Scope of reported evaluations . . . 16
3.7 Positioning of this thesis . . . 16

4 Deep Learning Architectures for Semantic Segmentation 18
4.1 Problem formulation and encoder-decoder principles . . . 18
4.2 Fully Convolutional Networks (FCN) . . . 19
4.2.1 Fully convolutional transformation . . . 19
4.2.2 Skip architecture and upsampling . . . 20
4.3 DeepLabV3 and atrous convolutions . . . 20
4.3.1 Atrous convolution formulation . . . 22
4.3.2 Atrous Spatial Pyramid Pooling (ASPP) . . . 22
4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head . . . 23
4.4.1 Depthwise separable convolutions . . . 23
4.4.2 Inverted residual blocks with linear bottlenecks . . . 24
4.4.3 Squeeze-and-Excitation (SE) channel attention . . . 25
4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid . . . 26
4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture . . . 26
4.5 Segment Anything Model 3 (SAM 3) for dataset annotation . . . 27
4.5.1 Architecture overview . . . 29
4.5.2 Operational distinction from onboard models . . . 31
4.6 Summary of architectural characteristics . . . 31

5 Dataset Preparation and Comparative Model Training 33
5.1 Image acquisition and class definition . . . 35
5.1.1 Image sources and acquisition conditions . . . 35
5.1.2 Semantic classes and labeling targets . . . 35
5.2 Dataset preparation . . . 35
5.2.1 Lettuce dataset with manual annotation . . . 35
5.2.2 SAM 3-assisted annotation for multi-crop datasets . . . 36
5.2.3 Dataset partitioning . . . 37
5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP . . . 37
5.3.1 Model selection and pretrained weights . . . 37
5.3.2 Training configuration . . . 38

5.3.3 Evaluation metrics and results . . . 40
5.3.4 Selection of LR-ASPP . . . 41
5.4 Supervision strategy comparison . . . 41
5.4.1 Training configurations and evaluation targets . . . 41
5.4.2 Evaluation results . . . 42
5.4.3 Selection of binary path configuration . . . 42
5.5 Model export and inference configuration . . . 42
5.5.1 ONNX export . . . 42
5.5.2 Inference inputs and outputs . . . 42

6 Geometric Extraction of the Local Navigation Reference 43
6.1 Input representation and edge-line models . . . 43
6.2 Region of interest and mask preparation . . . 44
6.3 Horizontal strips and candidate intervals . . . 45
6.3.1 Strip construction and column projection . . . 45
6.3.2 Local support around interval boundaries . . . 46
6.4 Initialization and bottom-up association . . . 46
6.4.1 Edge-support seed selection and gap-limited association . . . 46
6.5 Separate robust fitting of the corridor edges . . . 47
6.5.1 Excluding image-boundary observations . . . 47
6.5.2 Residual-based reweighting . . . 47
6.6 Quality indicators, validation, and image-space output . . . 48
6.6.1 Method 4: geometric checks and a composite score . . . 48
6.6.2 Interpretation of status and confidence . . . 49
6.7 Default configuration . . . 50
6.8 Effect of strip height and number . . . 50
6.9 Baseline algorithms for comparative evaluation . . . 51
6.9.1 Center-point strip tracking baseline . . . 51
6.9.2 Column-histogram baseline . . . 52
6.9.3 RANSAC-based contour baseline . . . 53
6.10 Comparative trade-offs and structural limitations . . . 53
6.10.1 Structural limitations of Method 4 . . . 54

7 Robot Platform and Integration 56
7.1 Robot hardware and sensors . . . 56
7.1.1 Robot platform . . . 56
7.1.2 Camera configuration . . . 57
7.1.3 Computing platform . . . 59
7.2 Software architecture . . . 59
7.2.1 ROS integration . . . 59
7.2.2 Perception pipeline implementation . . . 64

7.2.3 Control interface 64
7.3 Deployment configuration 65
7.3.1 Model export to ONNX 65
7.3.2 Real-time inference 65
7.3.3 Timing and latency 65
7.3.4 Software-in-the-Loop pre-validation 66

8 Experimental Validation 68
8.1 Offline evaluation protocol 68
8.1.1 Segmentation metrics 68
8.1.2 Geometric reference accuracy 70
8.2 Field validation 72
8.2.1 Ground-truth reference: laser sensor 73
8.2.2 Field evaluation metrics 73
8.2.3 Test conditions 75
8.2.4 Closed-loop navigation results 75
8.2.5 Qualitative observations 76
8.3 Discussion 76
8.3.1 Segmentation performance 76
8.3.2 Geometric extraction trade-offs 76
8.3.3 Limitations 77

9 Conclusions and Future Work 79
9.1 Main contributions 79
9.2 Conclusions on segmentation and annotation 80
9.3 Conclusions on geometric extraction 80
9.4 Limitations 81
9.5 Future work 82
9.5.1 Dataset and annotation 82
9.5.2 Segmentation and model deployment 82
9.5.3 Geometric extraction 83
9.5.4 System integration and field validation 83
9.6 Closing remarks 83

Bibliography 85

List of Figures

2.1 Agricultural inter-row environment, vehicle frames, and geometric error definition 6
2.2 Complete vision-based local guidance pipeline from camera stream to actuator commands 7

4.1 Fully Convolutional Network (FCN) architecture with skip connections 21
4.2 DeepLabV3 architecture featuring the Atrous Spatial Pyramid Pooling (ASPP) module with parallel atrous convolutions and image-level pooling (adapted from Chen et al. [10]). 23
4.3 MobileNetV3 Inverted Residual block and Lite Reduced ASPP (LR-ASPP) head 28
4.4 Original Segment Anything Model (SAM) architecture with ViT image encoder, prompt encoder, and mask decoder (adapted from Kirillov et al. [12]). SAM 3 extends this foundation with a dual detector-tracker architecture and open-vocabulary concept segmentation. 30

5.1 Dataset sample images and semantic ground-truth masks across crop species 34
5.2 SAM 3 zero-shot promptable annotation and mask refinement workflow 36
5.3 Training loss and validation mIoU progression across epochs for FCN, DeepLabV3, and LR-ASPP 39

6.1 Horizontal strip decomposition and column-wise projection for path interval detection 45
6.2 Robust line fitting of corridor boundaries and centerline construction 48

7.1 Physical mobile robot platform and front camera mounting setup . 58
7.2 ROS 2 software architecture node graph and topic communication flow 60
7.3 Execution time breakdown of the synchronous perception-control cycle on NVIDIA Jetson 66

8.1 Visual comparison of semantic segmentation outputs across crop types and network architectures 69
8.2 Comparative evaluation of geometric reference extraction across the five evaluated algorithms 71
8.3 Lateral tracking error and heading error profiles during inter-row autonomous navigation 75

Chapter 1

Introduction

▶ **DA COMPLETARE: Contesto di tesi e motivazione applicativa**

Inquadrare l'attività svolta nell'ambito della tesi di laurea magistrale:

- Collaborazione con il gruppo di ricerca di riferimento (Politecnico di Torino).
- Esigenze dell'agricoltura di precisione: diserbo meccanico selettivo, monitoraggio delle colture e riduzione dell'impatto ambientale senza dipendenza da segnali GNSS RTK sotto chioma o in serra.
- Obiettivo di realizzare una soluzione autonoma, modulare e fruibile su calcolatori embedded a basso costo (famiglia NVIDIA Jetson).

1.1 Context and motivation

Global population growth and rising food demand exert increasing pressure on agricultural productivity, while the sector faces persistent labor shortages and rising operating costs [1, 2]. Addressing these demands requires higher automation in field operations such as weeding, spraying, and crop monitoring. Row-based planting represents the dominant arrangement in modern farming, accounting for approximately 75% of cultivated land in structured agriculture [3]. Autonomous ground vehicles designed for these settings must navigate along inter-row corridors reliably without damaging crops or compacting productive soil beds.

Outdoor agricultural navigation historically relied on Global Navigation Satellite Systems (GNSS) augmented by Real-Time Kinematic (RTK) corrections. However, satellite positioning presents well-documented limitations in field environments: thick crop canopies cause multi-path reflections and severe signal attenuation, while greenhouse installations obstruct satellite reception entirely [1, 3]. Furthermore, RTK base station infrastructure introduces significant financial overhead. Alternative exteroceptive sensors such as 3D LiDAR provide direct spatial measurements,

but their high unit cost hinders economic scalability across commercial robotic fleets, and their point clouds lack the color and textural information required to distinguish crops from weeds [1, 3]. Monocular RGB cameras offer a cost-effective alternative that captures detailed chromatic and texture cues directly from the scene.

Early vision-based guidance pipelines extracted navigation lines through handcrafted image processing techniques. These approaches compute color vegetation indices to isolate plant matter from soil, apply intensity thresholding such as Otsu's method, and detect corridor edges using line-fitting algorithms or Hough transforms [1, 4]. Although computationally lightweight, these calibrated pipelines degrade when ambient field conditions shift. Variable sunlight angles, cast shadows from foliage, soil moisture gradients, specular reflections on wet leaves, and weed infestations frequently disrupt fixed thresholding rules and generate false boundary estimates.

Deep learning architectures, particularly convolutional neural networks for semantic segmentation, overcome these hand-tuning limitations. By learning hierarchical visual representations directly from image datasets, segmentation networks achieve higher tolerance to illumination shifts, ground irregularities, and weed clutter than handcrafted color models.

However, deep neural networks introduce a practical operational bottleneck: supervised training requires extensive pixel-level ground truth. Manually labeling agricultural scenes is labor-intensive and must cover multiple crop types, growth stages, and ambient conditions. In addition, running neural networks on compact onboard computers requires balancing segmentation accuracy against processing speed and power consumption, as steering commands must be updated fast enough to guide the vehicle in real time.

This thesis addresses these challenges through a modular pipeline that combines model-assisted dataset annotation, real-time onboard semantic segmentation, and deterministic geometric reference extraction for local robot guidance.

1.2 Objectives of the thesis

The main objectives of this work are:

- Compare manual and SAM 3-assisted annotation strategies for agricultural segmentation datasets spanning lettuce, tomato, and cucumber.

- Select a segmentation architecture balancing accuracy and inference speed for robotic deployment, through offline evaluation of FCN, DeepLabV3, and LR-ASPP on manually annotated lettuce images.

- Evaluate the effect of output label configurations (four-class, three-class, and binary path) on path segmentation accuracy.
- Implement and compare two edge-based geometric extractors that convert segmentation masks into local image-space navigation references.
- Integrate the selected perception and geometric components on a robot platform for field validation.

1.3 Organization of the work

The thesis is organized as follows:

Chapter 2 introduces the agricultural navigation problem, the role of vision-based local guidance, and the segmentation-to-control pipeline structure.

Chapter 3 reviews vision-based methods for agricultural row navigation, covering classical and learning-based segmentation approaches, geometric extraction techniques, and experimental validation in related work.

Chapter 4 details the mathematical formulations and designs of the deep learning architectures evaluated in this work, focusing on FCN, DeepLabV3, MobileNetV3 with the LR-ASPP head, and SAM 3.

Chapter 5 describes dataset preparation with manual and SAM 3-assisted annotation, empirical comparisons among the three architectures on lettuce, and evaluations of supervision strategies across label configurations.

Chapter 6 presents the two geometric extractors that convert corridor masks into navigation references through horizontal-strip interval selection and independent edge fitting.

Chapter 7 describes the robot hardware, camera configuration, software integration using ROS, and the deployment of the trained segmentation model.

Chapter 8 reports offline and field validation results for segmentation accuracy, geometric reference extraction, and closed-loop navigation performance.

Chapter 9 summarizes the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

Background and Problem Formulation

This chapter establishes the agricultural navigation context and formulates the vision-based local guidance problem addressed in this thesis.

2.1 Agricultural autonomous navigation

2.1.1 Navigation tasks and requirements

Agricultural robots perform tasks such as monitoring, spraying, harvesting, and weeding in structured crop environments [1]. Navigation between crop rows requires the robot to follow a path aligned with the row structure while avoiding plants and obstacles. The guidance reference must be computed from local sensing, as global GNSS positioning may be obstructed by crop canopies or greenhouse structures [1, 3].

Vision-based methods identify crop rows, inter-row paths, or crop boundaries from camera images and convert these observations into steering commands [1, 2]. The task is local: the robot maintains its position relative to the visible row structure, not a global map.

2.1.2 Environmental challenges

Field conditions vary with illumination, shadows, weather, growth stage, and weed presence [5, 2]. Plants may occlude parts of the path, and gaps or irregular spacing disrupt row structure. These changes affect both the appearance of image features and the reliability of geometric references extracted from them.

Segmentation models trained on one set of conditions must generalize to new lighting, crops, or row configurations. Geometric extractors must handle incomplete or ambiguous corridor observations without assuming perfect segmentation [6].

2.2 Vision-based local guidance

2.2.1 Sensing modalities

Camera-based perception provides appearance information for distinguishing plants, soil, and paths. Monocular RGB cameras are simpler and less expensive than LiDAR or stereo systems, but do not directly measure depth [1]. While RGB-D cameras provide both appearance and depth measurements, their performance depends on field conditions and hardware cost constraints.

This thesis uses monocular RGB sensing for local guidance. It does not reconstruct three-dimensional field structure or maintain a global map.

2.2.2 Image-space reference extraction

The guidance pipeline converts image observations into a reference suitable for control. Depending on the method, this may be a horizontal position, a fitted line, or a curve [7, 6, 4]. Image-space references describe the path geometry in pixel coordinates; they must be interpreted by the controller to generate velocity commands.

Straight-line models assume locally linear boundaries and produce simple references for proportional control [4]. They do not capture distant curves or complex row patterns, but are computationally efficient and suitable for short prediction horizons.

▶ DA COMPLETARE: Modello cinematico 4WS e formulazione analitica degli errori

Integrare la formalizzazione matematica del veicolo e del controllo:

- Modello cinematico per piattaforma 4WS a sterzatura simmetrica in controfase ($\delta_f = -\delta_r$) e relazione cinematica: $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega = \frac{2v\tan\delta}{L}$.
- Trasformazione rigida e proiettiva tra coordinate pixel (u, v) nel piano immagine e coordinate metriche nel frame robot $\{R\}$.
- Definizione rigorosa dell'offset laterale e_y e dell'heading error e_θ calcolati rispetto alla retta di mezzeria stimata.
- Formulazione della legge di guida proporzionale / Stanley per la determinazione della velocità angolare $\omega = K_p e_y + K_\theta e_\theta$.



Figure 2.1: Agricultural inter-row environment, vehicle frames, and geometric error definition

2.3 The segmentation-to-control pipeline

[SPAZIO FIGURA / SCHEMA]

Complete vision-based local guidance pipeline from camera stream to actuator commands

Schema a blocchi dell'architettura di navigazione: flusso continuo dall'acquisizione delle immagini monoculari, attraverso la rete di segmentazione semantica, l'estrazione geometrica dei bordi del corridoio, il calcolo degli errori di guida e la generazione dei comandi di velocità (v, ω).

Figure 2.2: Complete vision-based local guidance pipeline from camera stream to actuator commands

2.3.1 Perception stage

The perception stage assigns semantic labels to image pixels. For inter-row navigation, target classes typically comprise crop plants, the traversable path, covering material such as plastic mulch, and background [6]. The definition of output classes directly determines what the geometric stage can extract: a binary vegetation mask identifies crop rows, whereas a path mask isolates the drivable space between them [2, 3].

Segmentation models are trained on annotated field images. While transfer learning from pretrained backbones reduces training duration and improves feature generalization, the classification head must be tailored to the specific target crop classes and deployed at low latency on embedded hardware [3].

2.3.2 Geometric extraction stage

The geometric stage reduces the dense segmentation mask to a low-dimensional guidance reference for robot steering. Existing methods differ in their spatial search strategy and fitted geometric model: some aggregate mask pixels across image columns [4, 3], whereas others extract local intervals from horizontal strips [6] or search for line candidates within bounded angular regions [7].

The extraction algorithm must tolerate segmentation noise, missing plants, and cast shadows without producing erratic steering targets. Validation checks can reject estimates that fail geometric consistency thresholds, requiring the controller to handle transient frames where no valid reference is recovered [5].

2.3.3 Control integration

The controller converts the extracted geometric reference into forward velocity and steering commands. Proportional and look-ahead laws regulate steering using the lateral displacement and heading orientation errors relative to the image center [4].

In field operations, raw error signals extracted from semantic segmentation exhibit high-frequency jitter and occasional impulse disturbances caused by irregular foliage contours or transient detection dropouts. Directly feeding unfiltered errors into the steering loop induces actuator chattering and abrupt steering adjustments that compromise vehicle stability. Consequently, the control interface incorporates error conditioning structured across three complementary mechanisms:

1. **High-frequency smoothing:** a recursive low-pass filter attenuates persistent image-space tracking jitter without introducing excessive phase delay during row transitions;

2. **Impulse outlier rejection:** large step variations between consecutive frames are evaluated over temporal windows to isolate single-frame false detections while preserving genuine trajectory adjustments;

3. **Temporal hold and deadband:** a zero-order hold maintains command continuity across short-term perception dropouts, while a symmetric deadband around the corridor center prevents steering hunting.

The mathematical formulation and parameter tuning of this conditioning pipeline on the Demetra robotic platform are detailed in Section 7.2.1 of Chapter 7.

Closed-loop field evaluation measures the robot's ability to maintain row alignment under operational conditions. High segmentation accuracy and accurate geometric fitting are necessary prerequisites, but stable inter-row tracking ultimately depends on real-time execution and disturbance rejection over continuous runs [4, 2, 3].

Chapter 3

State of the Art

This chapter reviews vision-based methods for agricultural row navigation, focusing on the conversion of image observations into a local guidance reference. As surveyed by Bai et al. [1], vision sensors provide rich appearance and geometry cues for autonomous field guidance at low hardware cost. The review covers visual perception, geometric extraction, and the integration of these stages with robot control. Dataset preparation and the implementation used in this thesis are described in Chapters 5 and 6.

3.1 Classical image-processing methods

Classical pipelines combine image filtering, thresholding, and application-specific geometric rules. Mendez et al. convert vineyard images to grayscale, apply Gaussian filtering and Otsu thresholding, select the largest blob, and use column pixel counts to estimate the row center [4, Section 3.4]. Such methods can be effective in the conditions for which their parameters were selected, but changes in illumination, weeds, and crop appearance can require retuning [1].

3.2 Learning-based segmentation for agricultural navigation

3.2.1 Semantic segmentation approaches

Semantic segmentation assigns a class to each pixel, but the label definition determines what the geometric stage can recover. De Silva et al. train U-Net to predict narrow crop-row masks in sugar-beet images. Their labels represent row structure rather than individual plant pixels or traversable soil [2, Sections 3.3 and 4.1]. Navone et al. instead predict a binary vegetation mask from RGB images

using a MobileNetV3 backbone and an LR-ASPP head, then combine that output with depth for reference extraction [8, 3].

Like other path-oriented approaches in the literature, this work segments the traversable soil corridor between crop rows rather than the plants themselves. The segmentation model provides a candidate region of drivable ground, and the downstream geometric stage then extracts the lateral boundaries and computes the steering reference.

3.2.2 Training data and generalization

Pixel-level annotation is time-consuming, and agricultural datasets must represent changes in crop, growth stage, and lighting. De Silva et al. include 11 field variations in their dataset [2, Section 3]. Navone et al. initialize their segmentation model with ImageNet weights and train using synthetic AgriSeg images supplemented by 100 real crop images [3, Section 4.1, Table 1]. Pretraining and the choice of training images are separate design choices.

3.3 Semantic segmentation architectures

3.3.1 FCN, DeepLabV3, and mobile architectures

Fully Convolutional Networks (FCN) replace fully connected layers with convolutions to produce dense pixel-level predictions [9]. DeepLabV3 incorporates atrous convolutions and atrous spatial pyramid pooling to capture multi-scale context without decreasing feature resolution [10]. For embedded applications where compute resources are constrained, Howard et al. [11] introduced MobileNetV3 together with the Lite Reduced Atrous Spatial Pyramid Pooling (LR-ASPP) head, which reduces latency by using depthwise separable convolutions.

The choice of architecture affects both segmentation accuracy and inference speed. Deeper backbones or larger receptive fields can yield higher IoU values at the cost of additional compute. For robotic systems operating in closed loop, inference latency on edge hardware dictates the achievable control frequency.

3.3.2 Transfer learning and pretraining

Transfer learning initializes network weights from models pretrained on large visual corpora such as ImageNet. The pretrained backbone serves as a generic feature extractor, while the task-specific segmentation head is adapted to produce class scores matching the target dataset. In agricultural vision, pretraining reduces training time and mitigates overfitting when domain-specific annotated datasets are limited in size. For instance, Li et al. [6, Section 2.2.2] demonstrate that transferring

weights learned on corn fields to other crops (such as tomatoes, cucumbers, and wheat) preserves segmentation accuracy with small training sets. Similarly, Navone et al. [3, Section 4.1] initialize their segmentation backbone with ImageNet weights prior to fine-tuning on tree-row imagery.

The extent of pretraining differs across implementations. Some setups retain pretrained segmentation heads and replace only the final classification layer, whereas others initialize only the backbone and train the segmentation head from scratch.

3.4 Dataset preparation and model-assisted annotation

3.4.1 Manual annotation challenges

Manual pixel-level annotation requires annotators to label every pixel in training images. For agricultural datasets spanning multiple crops, growth stages, and lighting conditions, this process is time-consuming and expensive. Annotation consistency depends on annotator interpretation of ambiguous boundaries, particularly where plants overlap soil or covering material.

3.4.2 SAM and foundation models for segmentation

Foundation models for segmentation have established model-assisted annotation as an effective alternative to purely manual labeling. The Segment Anything project demonstrated that interactive models operating within a human-in-the-loop data engine can generate large-scale segmentation masks with minimal human intervention [12]. While early foundation models required spatial prompts such as bounding boxes or foreground/background clicks for every target instance, Segment Anything Model 3 (SAM 3) introduces open-vocabulary concept segmentation driven directly by textual descriptions and visual exemplars [13].

In precision agriculture, model-assisted annotation with foundation models has been explored primarily for satellite-based field boundary delineation and canopy phenotyping. However, its systematic application to agricultural ground navigation remains largely unexplored. In this thesis, SAM 3 is employed offline as a zero-shot concept annotator to construct multi-crop training datasets across lettuce, tomato, and cucumber rows, targeting specific navigation classes (soil corridors, crop canopies, and plastic mulch). This offline annotation pipeline decouples the heavy computational burden of foundation models from the real-time inference requirements of the onboard robot controller.

3.5 Geometric extraction methods

The geometric stage reduces a mask to a quantity the controller can use. Depending on the representation and method, the output may be a horizontal image position, a straight line, a crop-edge estimate, or a curve. These outputs describe different tasks and coordinate systems.

3.5.1 Line fitting and crop-edge detection

Chen et al. estimate a path line for greenhouse cucumber rows from navigation points selected in plant–soil segmentation. Their prediction-point Hough transform uses a least-squares estimate to restrict the line search and is compared with traditional Hough and least-squares fitting [7, Sections 2–3].

Kneip et al. detect crop–ground transitions in scanlines of a stereo elevation map, then fit a linear crop-edge model using a Huber M-estimator and iteratively reweighted least squares. Their output is a crop boundary in ground-plane coordinates, not an image-space estimate of the inter-row path [5, Sections 6–7].

3.5.2 Column-wise histogram methods

Histogram methods aggregate mask pixels by image column. Mendez et al. estimate the row center from the five columns with the largest pixel counts in a thresholded background region [4, Section 3.4.3]. Navone et al. use a different input: a temporally combined vegetation mask refined with depth. SegMin smooths its column histogram and selects the minimum; SegMinD weights the vegetation mask by normalized inverse depth before calculating the histogram [3, Section 3]. In both cases the output is a horizontal position, not a full path line.

3.5.3 Horizontal strips and adaptive multi-ROI methods

Li et al. combine a Faster-U-net inter-row-path mask with an adaptive multi-ROI procedure. Horizontal regions supply ordered local points, which are used as control points for a multisegment cubic B-spline. Their experiments use 40-pixel search regions and discuss the trade-off between the number of points and fitting behavior [6, Section 2.2.3, Figure 9].

This is the closest geometric comparison for the present work because both pipelines select local evidence from horizontal image regions. The representation and fitted model differ: Li et al. use an inter-row-path mask and a spline, whereas this thesis estimates a local straight reference from the soil corridor between parallel crop rows.

3.5.4 Triangle Scan Method

De Silva et al. propose the Triangle Scan Method (TSM) for crop-row masks. It first scans an upper horizontal band to find an anchor column, using a preset anchor if the scan fails a threshold. Candidate lines from that anchor to positions along the base of a triangular region are then evaluated; the line with the greatest mask support defines the central crop row. Its two endpoints provide position and angular errors for visual servoing [2, Section 4.2, Algorithm 1].

TSM is not the same as extracting one point from each of several horizontal strips: the upper scan establishes an anchor for a bounded line search. It also operates on crop-row masks, so its output is not directly equivalent to a reference extracted from a soil mask.

3.5.5 Bilateral boundary tracking versus centroid reduction

In agricultural vision pipelines that convert binary segmentations into geometric references, existing methods fall into distinct categories, each characterized by specific operational trade-offs:

1. **Crop-canopy feature extraction with RANSAC:** A classical paradigm detects feature points along crop canopies or plant stalks, partitions them into left and right row clusters, and fits linear models using Random Sample Consensus (RANSAC) combined with ordinary least squares [1]. Recent monocular vision implementations extract crop row anchor points via DBSCAN clustering followed by RANSAC fitting to generate navigation lines, with reported angular and lateral errors minimized on the central crop row [14]. While functional in dense, uniform vegetation, this approach depends on continuous plant growth and degrades when seedlings are missing or canopy density is irregular. In addition, RANSAC is non-deterministic: random hypothesis generation introduces frame-to-frame angular jitter between consecutive video frames, requiring downstream temporal filtering to stabilize steering commands. The stochastic nature of RANSAC stems from its iterative random sampling strategy: each iteration selects a minimal subset of points (typically two for line fitting), computes a candidate model, and counts inliers within a residual threshold. To achieve 99% confidence of finding an all-inlier sample for a two-point linear model when outliers constitute 50% of the data, the algorithm requires 16 iterations, rising to 35 iterations when the outlier ratio reaches 65%; this sample count grows exponentially as outlier contamination increases [15]. On embedded platforms operating at 20 Hz with strict latency budgets, the computational overhead and output variance of RANSAC present practical limitations for closed-loop control.

2. **Global contour tracking with line fitting:** An alternative approach applies edge operators (such as Canny filtering) or boundary tracing algorithms (`findContours`) over the full segmented corridor mask, separating the resulting pixel cloud into lateral boundaries before fitting lines with RANSAC. On dense and noisy boundary masks, this introduces variable computational latency and retains the stochastic jitter inherent to random sampling, confirming that raw RANSAC output on monocular imagery lacks the determinism required for smooth steering commands.

3. **Horizontal strip decomposition with centroid reduction:** Methods operating on segmented drivable soil corridors (such as the multi-ROI scheme of Li et al. [6]) partition the image into horizontal bands and extract the bounding interval $[x_L, x_R]$ within each strip. However, these methods immediately collapse each interval into its midpoint:

$$x_C = \frac{x_L + x_R}{2} \tag{3.1}$$

   fitting curves (such as cubic B-splines or polynomials) directly to the sequence of centroids x_C. This premature reduction discards the distinction between the two physical edges. Under asymmetric disturbances—such as protruding leaves, localized weeds, or shadows extending along one side of the corridor—the midpoint shifts toward the intrusion, which introduces systematic bias in both lateral offset and heading angle.

4. **Cascaded deep learning for line extraction:** Recent studies investigate two-stage deep architectures, where an initial segmentation model is followed by a secondary network dedicated to semantic line detection. While capable of learning line features directly from data, running two cascaded neural networks on embedded platforms exceeds the memory and latency limits required for closed-loop robotic control (≥ 20 Hz).

5. **Single-edge elevation fitting:** M-estimators have also been applied to crop boundaries, such as Huber-loss Iteratively Reweighted Least Squares (IRLS) on stereo elevation scanlines [5]. However, these formulations track a single cutting boundary along dense harvesting fronts, rather than parallel crop boundaries defining an inter-row corridor from monocular RGB imagery.

To address the trade-off between computational determinism and disturbance decoupling without cascading neural networks, this thesis investigates a decoupled dual-edge linear formulation:

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R \tag{3.2}$$

Both boundaries are estimated using Iteratively Reweighted Least Squares (IRLS) with a Cauchy loss function:

$$w_i = \frac{1}{1 + \left(\frac{r_i}{\sigma}\right)^2} \tag{3.3}$$

Because estimation operates on a discrete set of horizontal strip boundaries ($N \leq 24$), the IRLS solver converges deterministically within four iterations with $O(N)$ computational complexity (< 0.1 ms on NVIDIA Jetson Orin Nano). Asymmetric perturbations on one boundary alter only local weights for that boundary, without affecting the opposite boundary. The centerline reference $x_C(y) = (x_L(y)+x_R(y))/2$ is computed analytically only after both boundaries have converged independently.

3.5.6 Comparison of geometric representations

Table 3.1 summarizes the input representation and geometric output of the methods discussed above.

Table 3.1: Perceptual inputs and geometric references in selected agricultural navigation studies.

Study	Perceptual input	Geometric operation	Output
Chen et al. [7]	Plant–soil segmentation and navigation points	Restricted Hough search around a predicted point	Image-space path line
Kneip et al. [5]	Stereo elevation map	Scanline transitions and robust linear fit	Ground-plane crop edge
Mendez et al. [4]	Thresholded background region	Column counts and mean of the five largest	Horizontal row-center position
Navone et al. [3]	Vegetation mask with depth	Smoothed column histogram; inverse-depth weighting in SegMinD	Histogram-minimum position
Li et al. [6]	Predicted inter-row-path mask	Adaptive horizontal regions and cubic B-spline	Curved image-space path
De Silva et al. [2]	Predicted crop-row mask	Upper anchor and triangular line search	Central crop-row line
This thesis (Proposed DELE)	Predicted soil-corridor / mulch mask	Discrete horizontal strips, decoupled bilateral Cauchy-IRLS fitting	Straight image-space centerline $x_C(y)$ and (e_y, e_θ)

The methods are not directly ranked by this comparison: their inputs and outputs differ. Li et al. provide the closest structural precedent for strip-based

point selection, while TSM illustrates a line search for a crop-row representation. Neither method uses the same soil-corridor mask and straight-line model as the pipeline developed in this thesis.

3.6 System integration and experimental validation

3.6.1 Perception and control integration

Navigation systems use the extracted reference to generate motion commands. Navone et al. convert the displacement of their histogram reference into linear and angular velocity commands and smooth them with an Exponential Moving Average [3, Section 3.4]. Mendez et al. use proportional control based on row-center displacement and the distribution of background pixels [4, Section 3.5]. De Silva et al. combine the central row's angular and positional errors in a proportional visual-servoing controller for simulation [2, Section 4.3].

3.6.2 Scope of reported evaluations

The studies evaluate different stages of navigation. Kneip et al. assess crop-edge detection on recorded harvesting data [5, Section 8]; Chen et al. report angular errors and processing times for image-based path fitting [7, Sections 3.6, Tables 6–7]. De Silva et al. evaluate row detection on real images and visual servoing in simulation, with preliminary real-world row following also reported [2, Sections 5.2–5.3 and 6, Table 5]. Mendez et al. report closed-loop vineyard trials, measuring centering error from the distances to the two sides of the row [4, Sections 3.6 and 4]. Navone et al. test in simulation and crop fields, using LiDAR and attitude measurements to assess displacement and heading [3, Sections 4.3–4.4].

Segmentation IoU, image-space line error, field tracking error, and processing time measure different parts of the system. Results from these studies therefore provide context, not a common performance benchmark.

3.7 Positioning of this thesis

The system developed in this thesis addresses autonomous navigation in GNSS-denied crop rows using monocular RGB vision and embedded computation. Its technical positioning relative to the state of the art rests on three architectural decisions:

1. **Perceptual representation (Soil corridor versus canopy):** Rather than detecting crop stalks or foliage masks, the segmentation stage targets the

navigable soil corridor (and plastic mulch). This makes the navigation reference independent of crop growth stage, canopy height, and missing plants (fallanze), providing a continuous geometric guide directly at ground level.

2. **Geometric extraction (Decoupled deterministic boundaries versus centroid reduction):** Rather than collapsing strip intervals into midpoints prior to fitting (as in Li et al. [6]) or employing stochastic RANSAC over dense edge contours, the proposed pipeline decouples the left and right physical corridor boundaries. Each boundary is fitted independently using a Cauchy-weighted IRLS formulation that converges in four fixed iterations (< 0.1 ms). Unilateral disturbances (such as leaf overhangs or shadows) are suppressed locally on the affected boundary without distorting the opposite side or biasing the central heading angle.

3. **Embedded deployment feasibility:** The architecture decouples offline model-assisted dataset generation from onboard execution. Large foundation models (SAM 3) are restricted to offline zero-shot concept annotation, while real-time inference on the NVIDIA Jetson Orin Nano is handled by a lightweight LR-ASPP network (using a MobileNetV3 backbone) paired with the deterministic IRLS geometric extractor. The entire pipeline executes in closed loop at 20 Hz without requiring secondary neural networks, stereo elevation mapping, or LiDAR sensors.

The chapters that follow detail and evaluate each stage of this pipeline: Chapter 4 formulates the deep learning architectures, Chapter 5 analyzes the annotation protocols and comparative model training, Chapter 6 formalizes the geometric extractors, Chapter 7 describes the Demetra robotic platform and onboard ROS integration, and Chapter 8 reports offline benchmarks and field tests.

Chapter 4

Deep Learning Architectures for Semantic Segmentation

Semantic segmentation in agricultural robotics consists in assigning a predefined category label to every pixel of an acquired image. For autonomous row navigation, the perception system must distinguish navigable soil corridors from surrounding crop canopies, weeds, and field boundaries under variable outdoor illumination.

This chapter presents the mathematical formulation and architectural design of the deep learning models evaluated in this work. First, the segmentation task and encoder-decoder principles are defined. Second, Fully Convolutional Networks (FCN) and DeepLabV3 are detailed with emphasis on atrous convolutions and multi-scale context aggregation. Third, MobileNetV3 and the Lite Reduced Atrous Spatial Pyramid Pooling (LR-ASPP) head are analyzed, detailing the depthwise separable convolutions and channel attention mechanisms that reduce computational complexity for edge computing. Finally, the Segment Anything Model 3 (SAM 3) is described as an offline annotation engine for training label generation.

4.1 Problem formulation and encoder-decoder principles

Let $I \in \mathbb{R}^{H \times W \times 3}$ denote an input color image with height H and width W. Semantic segmentation aims to estimate a dense label map $Y \in \{1, \ldots, C\}^{H \times W}$, where each pixel (i, j) is assigned to one of C mutually exclusive semantic classes. Deep neural networks approximate this mapping through a parameterized function $f_\theta(I)$ that outputs an unnormalized score tensor $Z \in \mathbb{R}^{H \times W \times C}$. The posterior probability of

class c at pixel (i, j) is obtained via the spatial softmax function:

$$p_{i,j,c} = \frac{\exp(Z_{i,j,c})}{\sum_{k=1}^{C} \exp(Z_{i,j,k})}. \tag{4.1}$$

During supervised training with ground-truth one-hot masks $Y^* \in \{0,1\}^{H \times W \times C}$, parameters θ are optimized by minimizing the multi-class cross-entropy loss:

$$\mathcal{L}_{\text{CE}}(Y^*, p) = -\frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \sum_{c=1}^{C} Y^*_{i,j,c} \log(p_{i,j,c}). \tag{4.2}$$

Standard convolutional architectures for image classification progressively reduce spatial dimensions via strided convolutions and max-pooling layers. While spatial reduction increases the receptive field and aggregates contextual information, it discards fine-grained boundary positions. Semantic segmentation architectures solve this trade-off using an encoder-decoder topology:

- **Encoder (Backbone):** A sequence of convolutional stages that extracts high-level semantic features $F_{\text{enc}} \in \mathbb{R}^{\frac{H}{S} \times \frac{W}{S} \times C_{\text{enc}}}$, where $S \geq 1$ denotes the total output stride (typically $S \in \{8, 16, 32\}$).
- **Decoder (Segmentation Head):** A module that restores spatial resolution to $H \times W$ through bilinear interpolation or transposed convolutions, combining low-level spatial details from earlier encoder layers with abstract high-level features.

4.2 Fully Convolutional Networks (FCN)

Long et al. [9] introduced Fully Convolutional Networks by replacing the final dense (fully connected) classification layers of standard convolutional backbones with convolutional filters.

4.2.1 Fully convolutional transformation

In standard classification networks, the spatial feature map produced by the final convolutional layer is flattened into a vector and passed to dense layers, restricting the network to fixed input resolutions. FCN replaces each dense layer with a 1×1 convolutional layer whose kernel depth matches the dense weight matrix. For a feature map $F \in \mathbb{R}^{h \times w \times C_{\text{in}}}$, a convolutional layer with C filters of size 1×1 computes:

$$Z_{i,j,c} = b_c + \sum_{k=1}^{C_{\text{in}}} W_{c,k} \cdot F_{i,j,k}, \qquad \forall i \in \{1, \ldots, h\},\ j \in \{1, \ldots, w\}, \tag{4.3}$$

producing a coarse score map of dimension $h \times w \times C$. This allows the network to process inputs of arbitrary dimensions in a single forward pass without sliding-window recomputation.

4.2.2 Skip architecture and upsampling

To restore the coarse score map from stride $S = 32$ back to the original image dimensions $H \times W$, FCN employs transposed convolutions (also known as backward strided convolutions or fractionally strided convolutions). A transposed convolution with stride s places the input values on a dilated grid and computes the output via standard convolution, learning an upsampling filter $w \in \mathbb{R}^{K \times K}$:

$$\hat{y}_{i',j'} = \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} w_{u,v} \cdot x_{\lfloor i'/s \rfloor - u,\, \lfloor j'/s \rfloor - v}. \tag{4.4}$$

Upsampling directly from stride 32 produces blurry boundaries. Long et al. therefore designed skip connections:

- **FCN-32s:** Upsamples the final stride-32 score map directly by a factor of 32 in a single step.
- **FCN-16s:** Combines the 2× upsampled stride-32 score map with predictions extracted from the stride-16 stage (e.g., `conv4`), then applies 16× upsampling.
- **FCN-8s:** Combines the 2× upsampled FCN-16s score map with predictions extracted from the stride-8 stage (e.g., `conv3`), followed by an 8× upsampling step.

Implementation note: The TorchVision `fcn_resnet50` implementation adopted in this work uses a simpler architecture than the original FCN-8s. The segmentation head (FCNHead) processes the stride-32 features from ResNet-50's `layer4` with a 3×3 convolution (reducing channels by a factor of 4), batch normalization, ReLU activation, dropout (0.1), and a final 1×1 classification layer. An optional auxiliary classifier operates independently on `layer3` features for training supervision, but the main prediction head does not implement multi-stage fusion. Figure 4.1 illustrates the original FCN architecture with progressive refinement; the simplified TorchVision variant retains the encoder-decoder principle but omits intermediate score fusion.

4.3 DeepLabV3 and atrous convolutions

DeepLabV3, developed by Chen et al. [10], addresses the resolution loss caused by consecutive pooling operations by replacing standard downsampling with atrous (dilated) convolutions and multi-scale pooling modules.

[SPAZIO FIGURA / SCHEMA]

Fully Convolutional Network (FCN) architecture with skip connections

Schema a blocchi dell'architettura FCN: estrazione di feature a diversi stride dal backbone (ResNet-50) e upsampling tramite deconvolution combinando FCN-32s, FCN-16s e FCN-8s (da inserire da Long et al., 2015).

Figure 4.1: Fully Convolutional Network (FCN) architecture with skip connections

4.3.1 Atrous convolution formulation

Let $x[i, j]$ denote a 2D input feature map and $w[p, q]$ a convolutional filter of size $K \times K$ with $p, q \in \{-M, \ldots, M\}$ where $M = \frac{K-1}{2}$. Standard convolution samples adjacent positions on the input grid. Atrous convolution introduces a dilation rate $r \in \mathbb{N}^+$, defining the step with which the filter samples the input signal:

$$y[i, j] = \sum_{p=-M}^{M} \sum_{q=-M}^{M} x[i + r \cdot p,\ j + r \cdot q] \cdot w[p, q]. \tag{4.5}$$

When $r = 1$, Eq. (4.5) reduces to the standard convolution. For $r > 1$, the filter samples inputs with spaces of $r - 1$ zeros between consecutive filter taps. The effective filter size K' becomes:

$$K' = K + (K - 1)(r - 1). \tag{4.6}$$

For example, a 3×3 kernel ($K = 3$) with dilation rate $r = 6$ covers a spatial footprint of $3 + 2 \times 5 = 13 \times 13$ pixels, requiring only 9 learnable parameters. Atrous convolution expands the receptive field without adding parameters or reducing the spatial resolution of the feature maps.

4.3.2 Atrous Spatial Pyramid Pooling (ASPP)

Objects and agricultural corridors appear at varying visual scales depending on camera mounting angle and distance from the vehicle. DeepLabV3 captures contextual information across multiple scales using the Atrous Spatial Pyramid Pooling (ASPP) module.

The ASPP module operates in parallel on the high-level feature map extracted by the backbone and consists of five branches:

1. One 1×1 standard convolution with 256 filters and batch normalization.

2. Three 3×3 atrous convolutions with 256 filters each, using dilation rates $r \in \{6, 12, 18\}$ for an output stride of 16 (or $r \in \{12, 24, 36\}$ for stride 8).

3. One Image-Level Pooling branch: to incorporate global image context, this branch computes global average pooling across the spatial dimensions of the input tensor, processes the resulting $1 \times 1 \times C_{\text{in}}$ vector with a 1×1 convolution (256 filters) and batch normalization, and upsamples the output bilinearly to the original ASPP spatial resolution.

Implementation note: The TorchVision `deeplabv3_resnet50` implementation employs a dilated ResNet-50 backbone with output stride 8, achieved by

removing downsampling in `layer3` and `layer4` and introducing atrous convolutions. The ASPP module correspondingly uses dilation rates $r \in \{12, 24, 36\}$ for the three parallel atrous branches.

The outputs of all five branches are concatenated along the channel dimension, resulting in a tensor with $5 \times 256 = 1280$ channels:

$$F_{\text{concat}} = [F_{1\times1},\ F_{\text{atrous},r_1},\ F_{\text{atrous},r_2},\ F_{\text{atrous},r_3},\ F_{\text{image_pool}}]\,. \tag{4.7}$$

This concatenated representation is projected back to 256 channels via a 1×1 convolution with batch normalization and ReLU activation. The TorchVision implementation then applies a 3×3 convolution ($256 \to 256$ channels) with batch normalization and ReLU before the final 1×1 classification layer maps to C class logits. The output is upsampled via bilinear interpolation to image resolution $H \times W$. The ASPP parallel branch layout is depicted in Figure 4.2.

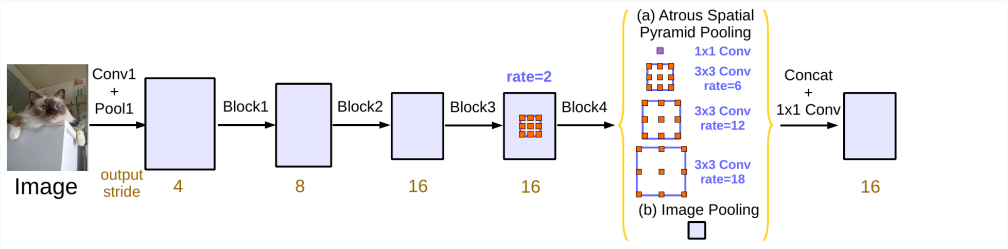


Figure 4.2: DeepLabV3 architecture featuring the Atrous Spatial Pyramid Pooling (ASPP) module with parallel atrous convolutions and image-level pooling (adapted from Chen et al. [10]).

4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head

While DeepLabV3 with a ResNet-50 backbone provides high segmentation accuracy, its 50.8 million parameters and heavy computational requirement result in high inference latency (86.87 ms per image), exceeding the real-time budget of mobile robot platforms. To achieve real-time operation on edge hardware, Howard et al. [11] developed MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) segmentation head.

4.4.1 Depthwise separable convolutions

The fundamental building block of MobileNet architectures is the depthwise separable convolution, which factors a standard convolution into two separate steps: a

spatial depthwise convolution and a channelwise pointwise convolution.

Consider an input feature map $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ and an output feature map $Y \in \mathbb{R}^{H \times W \times C_{\text{out}}}$, processed with spatial filter size $D_K \times D_K$.

- **Standard convolution:** Each filter channel interacts with every input channel simultaneously. The computational cost, measured in Multiply-Accumulate (MAC) operations, is:

$$\text{Cost}_{\text{standard}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2. \tag{4.8}$$

- **Depthwise convolution:** Applies a single $D_K \times D_K$ spatial filter per input channel independently, without channel cross-talk:

$$\text{Cost}_{\text{depthwise}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2. \tag{4.9}$$

- **Pointwise convolution:** Applies C_{out} filters of size 1×1 across all channels to compute linear combinations of the depthwise outputs:

$$\text{Cost}_{\text{pointwise}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.10}$$

The total cost of a depthwise separable convolution is the sum of Eqs. (4.9) and (4.10):

$$\text{Cost}_{\text{sep}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.11}$$

Comparing Eq. (4.11) to Eq. (4.8) yields the computational reduction ratio:

$$\frac{\text{Cost}_{\text{sep}}}{\text{Cost}_{\text{standard}}} = \frac{H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}}{H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2} = \frac{1}{C_{\text{out}}} + \frac{1}{D_K^2}. \tag{4.12}$$

For standard 3×3 filters ($D_K = 3$) and typical channel counts ($C_{\text{out}} \geq 64$), the reduction ratio is approximately $\frac{1}{9} \approx 0.11$. Depthwise separable convolutions thus reduce arithmetic operations and parameter counts by roughly 88–89% relative to standard convolutions.

4.4.2 Inverted residual blocks with linear bottlenecks

In classical ResNet-50 bottleneck blocks, the input channels are first compressed via a 1×1 convolution, processed with a 3×3 convolution at reduced dimensionality, and expanded back to the original high channel count (wide $\rightarrow$ narrow $\rightarrow$ wide). This structure reduces computational cost while maintaining representational capacity.

MobileNetV3 reverses this design using inverted residual blocks (narrow $\rightarrow$ wide $\rightarrow$ narrow):

1. **Expansion layer:** A 1×1 pointwise convolution expands the low-dimensional input C_{in} to a higher-dimensional space $e \cdot C_{\text{in}}$, where $e \in \{3, 6\}$ is the expansion factor.

2. **Depthwise layer:** A 3×3 or 5×5 depthwise separable convolution performs spatial feature extraction in the expanded channel space.

3. **Linear bottleneck:** A 1×1 projection layer compresses the channels back to C_{out}. Crucially, no non-linear activation function is placed after this final projection layer. Because non-linearities such as ReLU discard negative values by projecting them to zero, applying non-linear activations in low-dimensional spaces destroys useful geometric information. Retaining a linear bottleneck preserves manifold topology.

When the input and output channel dimensions match and the stride is 1, a residual shortcut connection adds the input directly to the block output.

4.4.3 Squeeze-and-Excitation (SE) channel attention

To adaptively calibrate channel dependencies, MobileNetV3 incorporates Squeeze-and-Excitation (SE) blocks within the inverted residual structure:

1. **Squeeze:** Global spatial average pooling aggregates the spatial dimensions of feature map $U \in \mathbb{R}^{H' \times W' \times C'}$ into a channel descriptor vector $z \in \mathbb{R}^{C'}$:

$$z_c = \frac{1}{H'W'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} U_{i,j,c}. \tag{4.13}$$

2. **Excitation:** Two fully connected layers parameterized by weights $W_1 \in \mathbb{R}^{\frac{C'}{4} \times C'}$ and $W_2 \in \mathbb{R}^{C' \times \frac{C'}{4}}$ model non-linear channel relationships with a channel reduction ratio of 4:

$$s = \sigma\left(W_2 \cdot \text{ReLU}(W_1 z)\right), \tag{4.14}$$

   where $\sigma(\cdot)$ denotes the hard-sigmoid activation function.

3. **Scale:** The input tensor U is rescaled channel-wise by the activation vector s:

$$\tilde{U}_{i,j,c} = s_c \cdot U_{i,j,c}, \qquad \forall c \in \{1, \ldots, C'\}. \tag{4.15}$$

4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid

The swish non-linearity, defined as $\text{swish}(x) = x \cdot \sigma(x)$, improves gradient flow in deep networks compared to standard ReLU. However, computing the exponential function $\sigma(x) = (1 + e^{-x})^{-1}$ introduces noticeable latency on embedded mobile processors lacking dedicated transcendental hardware units.

MobileNetV3 replaces standard sigmoid and swish with piece-wise linear hard approximations based on $\text{ReLU6}(x) = \min(\max(x, 0), 6)$:

$$\text{h-sigmoid}(x) = \frac{\text{ReLU6}(x+3)}{6}, \tag{4.16}$$

$$\text{h-swish}(x) = x \cdot \text{h-sigmoid}(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6}. \tag{4.17}$$

Eqs. (4.16) and (4.17) eliminate floating-point exponential operations, preserving the performance benefits of swish while minimizing execution time on embedded CPUs and GPUs.

4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture

Standard DeepLabV3 ASPP requires multiple parallel atrous branches with 256 channels each, resulting in significant memory bandwidth requirements. Howard et al. proposed the Lite Reduced ASPP (LR-ASPP) head as a lightweight alternative tailored for mobile segmentation.

The LR-ASPP architecture uses features from two distinct stages of the MobileNetV3-Large backbone:

- A low-resolution, high-semantic feature map $F_{\text{low}} \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times 960}$ extracted at output stride 16.
- A high-resolution, low-semantic feature map $F_{\text{high}} \in \mathbb{R}^{\frac{H}{8} \times \frac{W}{8} \times 40}$ extracted at output stride 8.

The TorchVision LR-ASPP implementation processes the two feature streams as follows:

1. The high-semantic, low-resolution feature map F_{low} passes through two parallel branches:
   - **CBR path:** A 1×1 convolution followed by batch normalization and ReLU activation projects F_{low} to an intermediate representation $X \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times C_{\text{inter}}}$:
   $$X = \text{ReLU}\left(\text{BN}\left(\text{Conv}_{1\times1}(F_{\text{low}})\right)\right). \tag{4.18}$$

- **Global context branch:** Applies global average pooling followed by a 1×1 convolution and a standard sigmoid activation to produce a channel-wise gating vector $G \in \mathbb{R}^{1\times1\times C_{\text{inter}}}$:

$$G = \sigma\left(\text{Conv}_{1\times1}\left(\text{GAP}(F_{\text{low}})\right)\right). \tag{4.19}$$

2. The CBR output X is modulated element-wise by the gating vector G and then upsampled by a factor of 2 via bilinear interpolation:

$$M_{\text{low}} = \text{BilinearUpsample}_{2\times}(X \odot G). \tag{4.20}$$

3. Both the modulated low-resolution features M_{low} and the high-resolution features F_{high} pass through **separate** 1×1 classification layers to produce class logits:

$$Z_{\text{low}} = \text{Conv}^{\text{high}}_{1\times1}(M_{\text{low}}), \qquad Z_{\text{high}} = \text{Conv}^{\text{low}}_{1\times1}(F_{\text{high}}). \tag{4.21}$$

4. The two logit maps are **summed in logit space** to produce the final stride-8 prediction:

$$Z_{\text{fused}} = Z_{\text{low}} + Z_{\text{high}}. \tag{4.22}$$

5. A final $8\times$ bilinear upsampling step restores the prediction to the original image dimensions $H \times W$.

Implementation note: Unlike architectures that fuse feature representations before classification, LR-ASPP fuses predictions in logit space. This design allows each branch to learn independent classification boundaries while combining their complementary spatial resolutions. The global context gating uses a standard sigmoid activation (Eq. (4.19)), not the hard-sigmoid approximation employed in the MobileNetV3 inverted residual blocks.

By replacing the multi-branch parallel ASPP structure with a single global pooling gate and logit-space fusion, LR-ASPP reduces computational operations while retaining global scene context and boundary localization. The LR-ASPP topology is summarized in Figure 4.3.

4.5 Segment Anything Model 3 (SAM 3) for dataset annotation

Manually annotating agricultural images at pixel precision requires considerable labor: drawing polygons around irregular plant boundaries, inter-row soil, and weed patches requires between 15 and 30 minutes per frame. Foundation models provide a mechanism to scale up annotation through zero-shot visual prompting.

[SPAZIO FIGURA / SCHEMA]

MobileNetV3 Inverted Residual block and Lite Reduced ASPP (LR-ASPP) head

Schema strutturale di MobileNetV3 con LR-ASPP: blocco Inverted Residual con Depthwise Separable Convolution, modulo Squeeze-and-Excitation (SE), e decoder LR-ASPP con ramo di global context gating e fusione delle feature a stride 8 (da inserire da Howard et al., 2019).

Figure 4.3: MobileNetV3 Inverted Residual block and Lite Reduced ASPP (LR-ASPP) head

Kirillov et al. [12] established the promptable segmentation paradigm with SAM, using Vision Transformers to generate binary segmentation masks from spatial prompts such as foreground/background points, bounding boxes, or coarse masks. Carion et al. [13] extended this paradigm in SAM 3 by introducing open-vocabulary concept segmentation driven by textual descriptions and visual exemplars.

4.5.1 Architecture overview

Unlike the original SAM architecture, which processes images through a single ViT encoder followed by a prompt-conditioned mask decoder, SAM 3 adopts a dual-architecture design comprising a **detector** for image-level segmentation and a **tracker** for video sequences. Both components share a unified **Perception Encoder (PE)** backbone that processes visual and linguistic inputs.

The SAM 3 detector follows a DETR-based paradigm with the following key components:

1. **Perception Encoder:** A shared vision-language backbone that encodes images into spatial feature representations. Unlike SAM's standalone ViT encoder, the PE is designed to handle both visual data and textual concept embeddings.

2. **Fusion Encoder:** Accepts unconditioned image embeddings from the PE and conditions them by cross-attending to prompt tokens derived from text descriptions (e.g., `"lettuce plant"`, `"navigable soil corridor"`) or visual exemplars (bounding boxes around reference instances).

3. **Presence Token:** A learned global token that decouples recognition from localization. The presence token predicts whether a specified concept exists in the image before attempting pixel-level segmentation, improving computational efficiency and reducing false positives.

4. **Decoder:** Learned object queries cross-attend to the conditioned image embeddings from the fusion encoder, producing a set of binary segmentation masks and associated confidence scores for all instances of the target concept.

For video input, SAM 3 extends this architecture with a memory-augmented tracker that shares the frozen PE backbone with the detector. The tracker incorporates a memory encoder and memory bank that store object appearance across frames, enabling consistent temporal segmentation.

Figure 4.4 illustrates the original SAM architecture, which established the foundation for prompt-based segmentation; SAM 3 extends this paradigm with text-driven concept detection and video tracking capabilities.

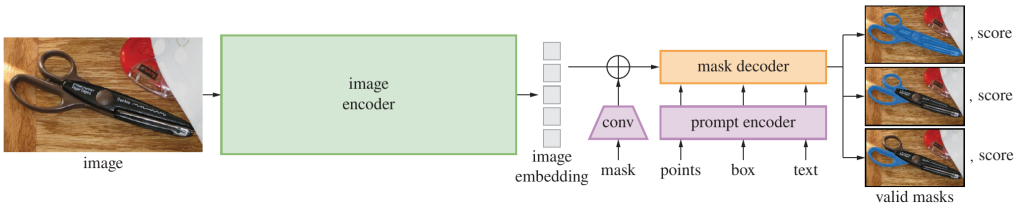


Figure 4.4: Original Segment Anything Model (SAM) architecture with ViT image encoder, prompt encoder, and mask decoder (adapted from Kirillov et al. [12]). SAM 3 extends this foundation with a dual detector-tracker architecture and open-vocabulary concept segmentation.

4.5.2 Operational distinction from onboard models

A key architectural distinction must be emphasized between the foundation model and the robotic segmentation network:

- SAM 3 requires over 600 million parameters and multi-gigabyte memory footprints, with single-image forward times exceeding 1000 ms on workstation GPUs. It cannot be deployed directly for closed-loop visual servoing on resource-constrained embedded computers such as the Jetson platform.
- In this thesis, SAM 3 functions strictly as an **offline supervision generator**. Given text prompts representing crop and corridor concepts, SAM 3 generates dense candidate masks across lettuce, tomato, and cucumber image collections.
- The resulting masks are reviewed, filtered, and used as ground-truth targets to train lightweight real-time architectures (FCN, DeepLabV3, and LR-ASPP). The distilled lightweight models execute inference on the robot platform in 10 to 86 ms, maintaining real-time control capability.

4.6 Summary of architectural characteristics

Table 4.1 summarizes the theoretical characteristics of the four architectures analyzed in this chapter, establishing the foundation for the dataset experiments and comparative evaluations presented in Chapter 5.

Table 4.1: Comparison of deep learning architectures evaluated in this thesis. Latency measurements correspond to inference on a single 512×512 RGB image using FP32 precision on the target embedded platform (details in Chapter 7).

Model	Backbone	Context mechanism	Target role	Primary characteristics
FCN	ResNet-50	Simplified segmentation head with auxiliary classifier	Benchmark segmentation	High parameter count, coarse boundary detail
DeepLabV3	ResNet-50	Multi-scale ASPP with stride-8 dilated backbone	High-accuracy benchmark	Highest computational cost, dense receptive field coverage
LR-ASPP	MobileNetV3-Large	Gated global pooling with logit-space fusion	Real-time robotic navigation	Compact architecture optimized for mobile deployment
SAM 3	Perception Encoder (vision-language)	DETR-based detector with presence token and text/visual prompts	Offline automatic annotation	Foundation model for open-vocabulary concept segmentation; not deployed onboard

Chapter 5

Dataset Preparation and Comparative Model Training

Following the theoretical analysis of deep learning architectures in Chapter 4, this chapter describes the datasets, annotation pipelines, and empirical training comparisons that guide the model configuration for robotic integration.

The experimental study began with an empirical comparison of FCN, DeepLabV3, and LR-ASPP on manually annotated lettuce images. Manual segmentation was time-consuming, so this first comparison was limited to lettuce rather than repeated across every target crop. Its outcome led to the adoption of LR-ASPP for embedded deployment.

SAM 3 was subsequently used to scale up dataset preparation, generating multi-crop training labels across lettuce, tomato, and cucumber collections. The resulting models were evaluated across alternative output label configurations: three classes (plant, path, background), four classes (including covering material), and a binary path-only representation.

This chapter details image acquisition, dataset partitioning, training parameters, and offline benchmark evaluations. The performance of the trained networks is evaluated separately from the annotation quality of the raw SAM 3 masks. Subsequent chapters detail geometric navigation extraction and closed-loop robot validation.



Figure 5.1: Dataset sample images and semantic ground-truth masks across crop species

5.1 Image acquisition and class definition

5.1.1 Image sources and acquisition conditions

▶ DA COMPLETARE: Origine delle immagini e condizioni di acquisizione

Specificare la sorgente dei dati e le modalità di raccolta:

- Dispositivo di ripresa utilizzato (risoluzione e ottica), quota di acquisizione e angolazione.
- Condizioni operative nei campi (aziende agricole o serre sperimentali, condizioni meteo e fasce orarie).
- Numero complessivo di immagini raccolte per lattuga, pomodoro e cetriolo.

5.1.2 Semantic classes and labeling targets

Three distinct labeling configurations were evaluated for SAM 3:

- **Four-class configuration:** Segments background, plants, soil, and covering material. This configuration was also used for the lettuce models during architecture selection.
- **Two-foreground-class configuration:** Labels plants and path, with background as the third output class. This produces three total semantic categories.
- **Path-only configuration:** Binary segmentation with background and path as the two output labels. For robotic integration, the path mask is the relevant output. Segmenting soil alone does not identify a unique navigation corridor or establish that it is obstacle-free. The geometric methods in Chapter 6 use the mask to construct a local image-space reference.

5.2 Dataset preparation

5.2.1 Lettuce dataset with manual annotation

The manually annotated lettuce dataset, identified as `Data`, contains 762 image–mask pairs distributed across eight folders. Its masks were prepared from the manual annotations stored in `annotations.json`. The same manual reference is used for the initial architecture comparison and for evaluating the models trained with SAM 3-generated multi-crop labels. Automatically annotated lettuce images belong to the latter training datasets, not to separate lettuce-only training configurations.

5.2.2 SAM 3-assisted annotation for multi-crop datasets

The time required for manual labeling motivated the use of SAM 3 to prepare the multi-crop datasets. Initial use suggested faster mask generation, but the total annotation effort has not been measured in a systematic comparison that includes inspection and correction.

The SAM 3-assisted labeling procedure covers lettuce, tomato, and cucumber images for the multi-crop training configurations. In this workflow, automatic segmentation is used to prepare labels offline; it is distinct from the segmentation model used during navigation.

For the lettuce images annotated with two foreground classes, the reported text prompts are `plants` and `central farm path`. The four-class labeling additionally uses `plastic mulch film` for covering material. Background accounts for the remaining label in each configuration. These reported lettuce prompts do not establish the exact prompts used for every crop in the multi-crop datasets. The binary configuration retains only the path annotation; crop-specific prompts and mask-composition settings for each configuration must be documented from the corresponding labeling runs.



Figure 5.2: SAM 3 zero-shot promptable annotation and mask refinement workflow

Tomato dataset

The tomato images were labeled using SAM 3 rather than the manual procedure used for the initial lettuce comparison. They contribute to the automatically annotated multi-crop training data. LR-ASPP was adopted after the comparison of the three candidate architectures on lettuce; no separate FCN, DeepLabV3, and LR-ASPP comparison was performed on tomato.

Cucumber dataset

SAM 3 was also used to label the cucumber images, which contribute to the automatically annotated multi-crop training data. As for tomatoes, the architecture choice followed the comparison on manually annotated lettuce, rather than a separate comparison on cucumber.

5.2.3 Dataset partitioning

The reported training runs use an 80% training, 10% validation, and 10% test partition with seed 42. The available `train_model.py` implements these proportions by assigning the integer parts of the training and validation counts and leaving the remaining samples for test. Applied to 762 pairs, this rule gives 609 training, 76 validation, and 77 test samples. These counts describe the intended manual-dataset partition; the saved indices and image lists must establish the actual membership of each evaluated split.

The test set is not used for gradient-based parameter updates. Checkpoint selection uses validation rather than test performance. A common seed alone does not ensure that differently sized or ordered datasets yield identical image partitions. Corresponding image identities must therefore be checked when comparing the manually supervised lettuce model with the automatically supervised multi-crop models, including whether a lettuce evaluation image appears in an automatic model's training set.

The reported operational evaluation selects 42 images from the manual test split belonging to `insalata_grande1`, `insalata_grande2`, and `insalata_media1`. These folders represent the row conditions selected for the robot application. The 42 images are a subset of the reported 77 test images, not an additional independent test set. Performance on this subset describes those selected conditions, rather than the full range of lettuce images.

5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP

This comparison addresses the architecture choice before changing the annotation procedure or crop coverage. FCN, DeepLabV3, and LR-ASPP were trained and evaluated using the manually annotated lettuce dataset.

5.3.1 Model selection and pretrained weights

FCN, DeepLabV3, and LR-ASPP were chosen from the implementations available in TorchVision. All three were trained on the manually annotated lettuce dataset.

Their selection was practical, rather than the outcome of a comparison with every available architecture.

The implementation separates each network into a backbone, which extracts image features, and a segmentation head, which produces class scores at each output location. Transfer learning is implemented by loading pretrained parameters and adapting the output layers to the target labeling task, rather than initializing every network parameter from scratch.

For FCN and DeepLabV3, the selected implementations use a ResNet-50 backbone with pretrained segmentation weights. After these weights are loaded, the final convolution of the main classifier is replaced by a new 1×1 convolution with one output channel per target class.

LR-ASPP is initialized differently. The implementation uses a MobileNetV3-Large backbone and requests its default pretrained weights through the `weights_backbone` argument. However, the `weights` argument of the complete segmentation model is set to `None`. Consequently, the backbone is pretrained, whereas the LR-ASPP segmentation head is newly initialized for the requested number of classes.

This distinction is relevant to the interpretation of the model comparison. The three configurations differ not only in architecture, but also in the extent of their pretrained initialization: FCN and DeepLabV3 retain pretrained segmentation-head layers, whereas LR-ASPP starts with a new segmentation head.

5.3.2 Training configuration

Training is organized into two stages. Before the first epoch, the backbone parameters are excluded from gradient-based updates by setting their `requires_grad` attribute to `False`. The complete main classifier remains trainable, as does the auxiliary classifier when present.

For the first ten epochs, AdamW is constructed using only the trainable parameters, with an initial learning rate of 10^{-3} and weight decay of 10^{-4}. A cosine-annealing scheduler is stepped after each epoch, with a minimum learning rate of 10^{-6}.

At the end of epoch ten, the script enables gradients for all backbone parameters. Fine-tuning of the backbone therefore begins at epoch eleven, together with continued training of the segmentation heads. A new AdamW optimizer is constructed with separate parameter groups: the backbone starts this stage at a learning rate of 10^{-5}, while the main and, where present, auxiliary classifiers start at 10^{-4}. Weight decay remains 10^{-4}.

A checkpoint is saved as `best.pth` whenever the validation loss is lower than all previous values; `last.pth` stores the latest training state. Selection therefore uses minimum validation loss, not maximum test IoU or simply the last epoch.

[SPAZIO FIGURA / SCHEMA]

Training loss and validation mIoU progression across epochs for FCN, DeepLabV3, and LR-ASPP

Curve di apprendimento durante le epoche di addestramento: andamento della Cross-Entropy Loss e della Mean IoU su set di validazione per le tre architetture candidate sul dataset di lattuga.

Figure 5.3: Training loss and validation mIoU progression across epochs for FCN, DeepLabV3, and LR-ASPP

5.3.3 Evaluation metrics and results

The comparison uses the three CSV records stored in `training insalata`: the FCN, DeepLabV3, and LR-ASPP `*_validation_metrics.csv` exports. Each contains aggregate metrics, the checkpoint epoch, and IoU values for background, plants, soil, and covering material. The reported mIoU agrees with the arithmetic mean of these four class IoUs, including background.

Table 5.1 reports the exported metrics. Inference times have been converted from seconds to milliseconds. The FPS values equal the reciprocal of the mean recorded time; they are not measurements of the complete robot processing or control rate. Checkpoint epochs identify the evaluated weights, not training duration.

Table 5.1: Architecture comparison from the manual lettuce validation exports. Timing values are those recorded in the CSV files, not end-to-end robot latency.

Model	Epoch	mIoU	Path IoU	Mean time (ms)	FPS
FCN	38	0.9232	0.9621	58.24	17.17
DeepLabV3	43	0.9228	0.9651	86.87	11.51
LR-ASPP	194	0.9108	0.9546	10.41	96.07

FCN and DeepLabV3 achieve mIoU values of 0.9232 and 0.9228, respectively, compared with 0.9108 for LR-ASPP. Using the unrounded CSV values, the LR-ASPP deficit is 1.24 percentage points relative to FCN and 1.19 relative to DeepLabV3. Its mean recorded time is 10.41 ms, compared with 58.24 ms and 86.87 ms. These timings correspond to a 5.60-fold reduction relative to FCN and an 8.35-fold reduction relative to DeepLabV3.

Table 5.2 provides the class-level comparison. LR-ASPP has lower IoU than both alternatives in each class, although its soil/path IoU remains 0.9546. The data support a trade-off between accuracy and latency, not a claim of equal segmentation accuracy.

Table 5.2: Per-class IoU from the same manual lettuce validation exports. Plants, soil, and covering material correspond to `pianta`, `suolo`, and `copertura` in these files.

Model	Background	Plants	Soil/path	Covering material
FCN	0.9464	0.9280	0.9621	0.8565
DeepLabV3	0.9464	0.9277	0.9651	0.8519
LR-ASPP	0.9394	0.9120	0.9546	0.8372

5.3.4 Selection of LR-ASPP

LR-ASPP was adopted because the substantial reduction in recorded inference time justified accepting an mIoU approximately 1.2 percentage points below the other two configurations. It was not the most accurate model, but offered the practical accuracy–latency compromise sought for robot perception. FCN and DeepLabV3 had higher IoU but substantially longer recorded inference times. Subsequent SAM-supervised multi-crop training therefore used LR-ASPP while varying the output label set.

This choice is based on the available offline comparison. The records do not establish a hardware-independent speed ranking or demonstrate that the integrated robot meets its control timing requirements. Those requirements must be assessed using the deployed model and the complete perception and control pipeline.

5.4 Supervision strategy comparison

5.4.1 Training configurations and evaluation targets

Table 5.3 summarizes the four training configurations considered in this comparison. This study is separate from the initial comparison of the three candidate architectures. The manual configuration is trained on lettuce only, whereas all three SAM-supervised configurations use multi-crop data and differ in their output label sets.

Table 5.3: Training scope and target labels for the comparison of manual and SAM 3-generated supervision. Class counts include background; evaluation uses the manual lettuce reference.

Configuration	Training scope	Output labels
Manual	Lettuce only (`Data`)	Background, plants, path, covering material
SAM-generated, two foreground classes	Multi-crop	Background, plants, path
SAM-generated, four classes	Multi-crop	Background, plants, path, covering material
SAM-generated, binary (`allPlantsSAM`)	Multi-crop	Background, path

Manual masks supply the ground truth for all comparisons. For path-only scoring, the target is the manual soil class and the predicted positive region is the corresponding path output of each model.

5.4.2 Evaluation results

Results to complete after provenance checks: Present the four-class, two-foreground-class, and path-only comparisons on the documented lettuce evaluation set. Report per-class IoU and state which classes each average includes. Add a path-focused summary, identifying the checkpoint and split for each configuration.

5.4.3 Selection of binary path configuration

The binary configuration was selected because its target is directly aligned with path extraction and its observed behavior on the selected operational images supported that choice. The quantitative evidence and limitations are to be completed in this section. This is not a claim that the binary configuration outperforms the manually supervised model or that performance on the selected rows holds across the entire dataset.

5.5 Model export and inference configuration

5.5.1 ONNX export

▶ DA COMPLETARE: Procedura di esportazione del modello in formato ONNX

Documentare i parametri dello script Python di export:

- Versione di PyTorch e versione Opset di ONNX utilizzata.
- Definizione della forma del tensore d'ingresso (batch fisso a 1, 3 canali colore, altezza e larghezza $H \times W$).
- Inclusione o esclusione della testa di softmax/argmax nel grafo computazionale del modello ONNX esportato.

5.5.2 Inference inputs and outputs

▶ DA COMPLETARE: Interfaccia di inferenza e specifiche dei tensori

Descrivere le specifiche del tensore d'ingresso (range di normalizzazione $[0, 1]$ o normalizzazione con media e deviazione standard di ImageNet) e del tensore di uscita (mappa delle probabilità o indici di classe discreti a risoluzione originale).

Chapter 6

Geometric Extraction of the Local Navigation Reference

This chapter describes the geometric extractor implemented in `pf4.py` (referred to as Method 4 in the experimental evaluation). It converts a segmented corridor mask into a local navigation reference by extracting intervals in horizontal strips, associating their left and right boundaries from bottom to top, and fitting each edge independently with a robust linear estimator. The centerline reference is the analytic mean of the two fitted edge lines.

6.1 Input representation and edge-line models

Both functions expect a two-dimensional label mask $M(y, x)$ of height H and width W. Image coordinates increase to the right in x and downward in y. The argument `path_class` identifies the class from which corridor intervals are extracted; its default is 1 in both files. A binary input can therefore use 1 for the target region and 0 elsewhere. For a multiclass segmentation mask, this argument must match the actual label encoding, or the mask must first be converted appropriately. The extractors do not infer the class mapping or use a separate plant class. A target-region label is not, by itself, a guarantee of traversable space.

The left and right boundaries are represented by

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R. \tag{6.1}$$

Treating x as a function of y permits vertical image lines without an infinite slope. The center reference is

$$x_C(y) = \frac{x_L(y) + x_R(y)}{2} = m_C y + q_C, \qquad m_C = \frac{m_L + m_R}{2}, \quad q_C = \frac{q_L + q_R}{2}. \tag{6.2}$$

Thus, both methods assume locally straight image-space boundaries. The models do not reconstruct terrain or estimate a metric trajectory. Their residuals and width estimates are in pixels, not lateral distances in meters. The two edges need not be parallel: their separation can change linearly with image height.

At each image height, the reference lies halfway between the two predicted boundaries. This construction is not generally equivalent to fitting the observed interval midpoints: the two edge fits can exclude different border-touching samples and assign different residual weights. Keeping the boundaries separate allows each visible side to contribute its own observations, although both sides must have enough samples.

The shared processing stages are:

1. select a lower region of interest and clean the target-class mask;
2. extract and merge projected intervals in horizontal strips;
3. select a seed using the method-specific support and priority rules;
4. associate interval boundaries upward through the image;
5. fit the non-clipped left and right boundaries independently;
6. calculate quality indicators and rasterize the center reference, with method-specific validation and output rules.

6.2 Region of interest and mask preparation

The ROI begins at $y_0 = \lfloor \rho H \rfloor$ and extends to the bottom of the image. The default starting ratio is $\rho = 0.30$ for Method 4. The binary target mask is

$$B(y, x) = \mathbf{1}\{M(y_0 + y, x) = \texttt{path_class}\}, \qquad 0 \le y < H_R = H - y_0. \tag{6.3}$$

Opening with a 5×5 elliptical kernel is applied first, then closing with a 9×9 rectangular kernel. Connected components with fewer than 500 pixels are removed; all sufficiently large components are retained, not only the largest one.

Let $\widetilde{B}$ denote the cleaned binary ROI mask. It supplies both interval extraction and local support calculations. These support calculations do not use the original multiclass mask to compute separate soil and plant ratios.

Morphology and interval merging can bridge interruptions or change region connectivity. Consequently, support in $\widetilde{B}$ measures agreement with the processed mask, not independently verified scene geometry. Kernel sizes, component areas, and distance thresholds are in input-mask pixels and depend on image resolution and camera configuration.

6.3 Horizontal strips and candidate intervals

6.3.1 Strip construction and column projection

Both methods use $N = 24$ strips by default, indexed from bottom to top. Their height is $h = \max(1, \lfloor H_R/N \rfloor)$. For strip index s, the ROI-relative bounds are

$$y_s^- = \max(0, H_R - (s+1)h), \qquad y_s^+ = \min(H_R, H_R - sh), \tag{6.4}$$

with the lower image row bound y_s^+ excluded. Empty strips are skipped. The representative full-image coordinate is $y_s = y_0 + (y_s^- + y_s^+)/2$, following the code's bound-based convention. Integer division may leave a remainder above the constructed strips.

For example, with $H_R = 157$ and $N = 24$, the strips cover the lowest $24 \times 6 = 144$ ROI rows. The upper 13 ROI rows do not supply candidates; the ROI and the region actually sampled by the strips are therefore not identical.

Within each strip, the cleaned mask is summed vertically. A column is active whenever its sum exceeds zero, so even one target pixel activates that column. Contiguous active columns form raw intervals. Neighboring intervals are merged when the number of inactive columns between them is at most `max_interval_gap`, whose default is 30 pixels. For a merged interval $[\ell_i, r_i]$, the width is

$$d_i = r_i - \ell_i + 1. \tag{6.5}$$

Intervals narrower than 8 pixels are discarded. There is no fixed maximum interval width in either new method. Instead, an interval touching both image margins is rejected if the mean of the entire cleaned binary strip is below 0.70. With `border_margin`=2, the flags are

[SPAZIO FIGURA / SCHEMA]

Horizontal strip decomposition and column-wise projection for path interval detection

Schema illustrativo della decomposizione della maschera binaria di navigazione in $N = 24$ strisce orizzontali e relativa proiezione delle colonne attive per l'identificazione degli intervalli di corridoio.

Figure 6.1: Horizontal strip decomposition and column-wise projection for path interval detection

$$t_{L,i} = \mathbf{1}\{\ell_i \leq 2\}, \qquad t_{R,i} = \mathbf{1}\{r_i \geq W - 1 - 2\}. \tag{6.6}$$

These flags indicate possible truncation by the image boundary, rather than verified physical corridor edges.

Column projection merges evidence across different vertical locations; a merged interval can also contain horizontal gaps. Neither its width nor its endpoints prove that its rectangular interior is entirely target class. This matters when interpreting intervals around occlusions or fragmented segmentation.

6.3.2 Local support around interval boundaries

Method 4 computes $u_{L,i}$ and $u_{R,i}$ in separate bands centered on ℓ_i and r_i. The effective half-width is $\max(2, \texttt{local_band_half_width})$, equal to 5 by default. These means quantify target-pixel occupancy around each boundary; they are not boundary-detection probabilities. Because a band straddles a boundary, it need not be entirely target class even for a clean interval.

Edge support ranks seed candidates and contributes to the final quality score. It is not an acceptance threshold and does not weight the line fit. In particular, `min_edge_support`=0.20 is present in the function signature but unused in the active implementation.

6.4 Initialization and bottom-up association

6.4.1 Edge-support seed selection and gap-limited association

Method 4 pools all candidates in the first six constructed strips from the bottom and selects the seed by maximizing $\min(u_{L,i}, u_{R,i})$, then interval width. No minimum-support threshold is applied, so even a weakly supported candidate can initialize tracking.

The boundary displacement δ_i and association cost are

$$D_4 = \max(28, 0.5 d_{\text{prev}}), \qquad J_{4,i} = \delta_i + 0.4|d_i - d_{\text{prev}}|. \tag{6.7}$$

Only the displacement limit determines admissibility; edge support does not enter this cost or an additional tracking gate.

A counter records consecutive strips without a compatible candidate and resets after each successful association. With `max_gap_strips`=3, tracking stops on the fourth consecutive miss. Stopping does not directly return `lost`: the accumulated points proceed to fitting if at least three are available. The maximum miss count,

including the miss that triggers stopping, enters the quality score. The previous interval remains unchanged during gaps.

The stopping rule and gap penalty have different effects: three misses are tolerated for continued association, but already reduce the gap score to zero. A fourth miss stops the search without automatically rejecting a reference supported by the observations collected earlier.

Method 4 performs greedy spatial association within one frame, not temporal tracking. Each accepted interval contributes two boundary observations, and at least three accepted intervals are required before the edge-fitting stage.

6.5 Separate robust fitting of the corridor edges

6.5.1 Excluding image-boundary observations

The left fit uses only accepted intervals not flagged as touching the left image margin. The right fit independently excludes intervals flagged as touching the right margin. At least two observations per edge must remain. The two fits can therefore use different strips and span different vertical ranges. A clipped boundary is not replaced by a one-sided centerline estimate: if either edge lacks sufficient samples, both implementations return an empty output with `lost` status.

The retained samples are interval endpoints from the cleaned column projection. They are not points obtained by a dedicated edge detector. Excluding border-touching samples reduces the influence of image truncation, but does not establish the correctness of the remaining boundaries.

6.5.2 Residual-based reweighting

Both methods start each edge fit with uniform weights. There is no semantic, interval-width, or vertical-position base weighting. For an edge sample (x_i, y_i), four reweighting iterations are performed by default, with scale $\sigma = 18$ pixels:

$$e_i^{(t)} = x_i - m^{(t)} y_i - q^{(t)}, \qquad w_i^{(t+1)} = \frac{1}{1 + (e_i^{(t)}/\sigma)^2}. \tag{6.8}$$

Each iteration calls `numpy.polyfit` with degree one and updated weights. Its weights multiply unsquared residuals [16]; the fitted objective at that iteration is

$$(m, q) = \arg\min_{m,q} \sum_i \left[w_i (x_i - m y_i - q) \right]^2 . \tag{6.9}$$

Thus, the effective coefficient of each squared residual is w_i^2, not w_i. The implementation should not be interpreted as a different standard robust-loss estimator

solely from the form of its weight rule. Weights are recomputed from the current model rather than accumulated across iterations; no convergence-based stopping criterion is used.



Figure 6.2: Robust line fitting of corridor boundaries and centerline construction

The final residual for each retained sample is its absolute horizontal deviation from the corresponding fitted edge. Define $\bar{e}$ as the mean over the concatenated left and right residual arrays, and

$$F = \text{clip}(1 - \bar{e}/\sigma,0,1). \tag{6.10}$$

Edges with more retained samples contribute more terms to this mean. Reweighting reduces the influence of inconsistent points but cannot correct an erroneous association chain that supports a coherent wrong line. It does not impose parallel edges or estimate genuine path curvature.

6.6 Quality indicators, validation, and image-space output

6.6.1 Method 4: geometric checks and a composite score

Method 4 validates the fitted boundaries before constructing the output. At 16 evenly spaced heights covering the combined range of retained edge samples, it evaluates

$$D(y) = x_R(y) - x_L(y). \tag{6.11}$$

All sampled widths must be strictly positive. The median of these sampled widths is returned as `width`. This check concerns the sampled observation range, not any extrapolation below it.

A separate indicator uses all accepted intervals, including those with border-touching endpoints. For K intervals in association order, let $z_j = r_j - \ell_j$; unlike extraction width d_j, this quantity omits the inclusive-endpoint +1. The width-irregularity indicator is

$$E_D = \frac{Q_{0.90}\left(\{|z_{j+2} - 2z_{j+1} + z_j|\}_{j=1}^{K-2}\right)}{\max(\text{median}_j\,|z_j|,1)}, \qquad A = \text{clip}(1 - E_D/0.30,0,1), \quad (6.12)$$

where $Q_{0.90}$ is the 90th percentile. These are second differences in sample order, not derivatives normalized by vertical distance. If strips are skipped, sample spacing is unequal; even a width varying linearly with image height need not yield zero second differences. The indicator therefore measures an implementation-specific irregularity, not a perspective-invariant physical curvature.

Let S be the mean of the left support values used by the left fit and the right support values used by the right fit, pooled together. With maximum consecutive miss count $g_{\max}$ and allowed gap $g_a = 3$,

$$G = \text{clip}(1 - g_{\max}/\max(g_a,1),0,1), \qquad C_4 = 0.25T + 0.20S + 0.25F + 0.20A + 0.10G. \quad (6.13)$$

The result is `lost` if the positive-width check fails, $\bar{e} > 2\sigma$, or $C_4 < 0.38$. Otherwise, it is `degraded` for $C_4 < 0.60$ and `ok` for $C_4 \geq 0.60$.

A result marked `lost` returns a zero output matrix before rasterization, with no centerline model in the post-fit rejection branch. For an accepted result, the mean edge line is rasterized from the top of the highest accepted strip to the bottom of the image, then dilated with a 3×3 elliptical kernel. This can extrapolate below the seed. The edge-order check does not cover the entire rendered range when that range exceeds the sampled observation range.

Diagnostics report the edge and centerline models, mean residual, support, tracking fraction, median fitted width, width irregularity, maximum gap, row bounds, and cleaned ROI mask. Earlier failures return `lost` with reason strings such as `no_seed_candidate`, `tracking_failed`, or `insufficient_edge_points`.

6.6.2 Interpretation of status and confidence

The confidence score C_4 is a heuristic combining processed-mask edge support with geometric and gap indicators. It is not a calibrated probability. A downstream controller must inspect the returned status; neither function specifies a robot stopping or recovery policy. Malformed inputs, unsuitable parameter values, or numerical exceptions are not comprehensively handled by the explicit failure branches.

6.7 Default configuration

Table 6.1 lists the default parameter values for Method 4. These are the defaults of `pf4.py`, not a statement that a controller uses them or that they are experimentally optimal. Overrides and mask encodings must be recorded for each experiment.

Table 6.1: Default configuration of Method 4 (`pf4.py`). Distances and areas are expressed in input-mask pixels.

Setting	Value
Target-class identifier	1
ROI starting ratio ρ	0.30
Number of strips N	24
Opening / closing kernels	5×5 / 9×9
Minimum component area	500 px
Minimum interval width	8 px
Maximum merged gap	30 px
Image-border margin	2 px
Full-strip occupancy gate	0.70
Local support-band half-width	5 px
Seed search window	6 strips
Jump-limit lower bound	28 px
Width-change cost coefficient	0.4
Consecutive misses allowed	3
Residual scale σ / iterations	18 px / 4
Width-error score scale	0.30
Lost / acceptable score thresholds	0.38 / 0.60
Output dilation kernel	3×3

The two-line representation retains boundary information separately, but requires sufficient non-clipped samples on both sides. Independent edge fits may be supported over different height ranges, and their mean can include extrapolation of one or both boundaries. A wide or well-supported interval is not necessarily the intended row.

6.8 Effect of strip height and number

The strip count controls vertical sampling and the maximum number of interval observations. More strips provide thinner vertical regions and potentially more points per boundary fit. Taller strips aggregate more mask pixels but mix evidence from different image heights, and column projection can conceal interruptions.

Increased strip height does not guarantee more reliable boundary estimates or a numerically better fit.

For a 224×224 input with $\rho = 0.30$, Method 4 has $y_0 = 67$ and $H_R = 157$. Table 6.2 shows strip heights for different strip counts using these dimensions. At $N = 24$, the strip height is 6 pixels. A fit can use fewer than N points because of seed location, missed strips, early stopping, or border-point exclusion; the two edges can have different sample counts.

Table 6.2: Strip heights for an illustrative 224×224 mask with $\rho = 0.30$ ($H_R = 157$). The point count is an upper bound per edge, not a guaranteed number of fitted samples.

Strips N	Strip height (px)	Max. edge samples
24	6	24
20	7	20
16	9	16
12	13	12
8	19	8

Changing N also changes the vertical extent of a fixed six-strip seed window and the extent of a three-strip tolerated gap. These settings should therefore be considered together. The width-irregularity indicator depends on the sampling sequence, while tracking fractions use the number of constructed strips as their denominator. Selecting N requires evaluating boundary and centerline errors, accepted-point counts, rejection rates, and processing time under the intended image resolution and field conditions. No optimal strip count is inferred from the implementation alone.

6.9 Baseline algorithms for comparative evaluation

To quantify the performance of decoupled dual-edge regression against alternative geometric strategies, three baseline extractors are integrated into the evaluation pipeline: a center-point strip tracker, a column-histogram extractor, and a RANSAC-based dual-edge extractor.

6.9.1 Center-point strip tracking baseline

The center-point strip tracker (`pf_centri.py`) shares the pre-processing stage, region of interest ($y_0 = \lfloor 0.30H \rfloor$), and horizontal strip decomposition ($N = 24$) of

Method 4. In each strip, horizontal intervals are extracted from the binary corridor mask. Instead of retaining the left and right boundary coordinates independently, the algorithm computes the midpoint and width of each candidate interval:

$$x_C = \frac{x_L + x_R}{2}, \qquad w = x_R - x_L + 1. \tag{6.14}$$

Tracking starts from the candidate interval with the highest pixel density within the bottom six strips. Successive intervals are associated bottom-up by minimizing a transition cost combining lateral center displacement and corridor width variation:

$$J_{\text{center}} = |x_C^{(k+1)} - x_C^{(k)}| + 0.4\,|w^{(k+1)} - w^{(k)}|, \tag{6.15}$$

subject to a maximum allowable jump $|x_C^{(k+1)} - x_C^{(k)}| \leq \max(28, 0.5\,w^{(k)})$.

Once tracking reaches the upper strip or exceeds the maximum gap of three consecutive empty strips, a single centerline model $x_C(y) = m_C y + q_C$ is fitted to the collected centers $(y_k, x_{C,k})$ using IRLS with Cauchy weighting across four iterations. Because interval midpoints are formed prior to fitting, asymmetric border disturbances shift $x_{C,k}$ directly, transferring localized boundary noise into the fitted trajectory.

6.9.2 Column-histogram baseline

The column-histogram baseline (`pf_histogram.py`) reproduces the SegMin methodology developed by Navone et al. [3]. It operates globally over the lower ROI of the segmentation mask without strip subdivision.

Pixel values corresponding to the traversable path class are accumulated along each image column:

$$H(x) = \sum_{y=y_0}^{H-1} M(y, x), \qquad x \in [0, W-1]. \tag{6.16}$$

The raw profile $H(x)$ is smoothed using a one-dimensional moving-average filter to suppress high-frequency boundary irregularities:

$$\tilde{H}(x) = \frac{1}{2k_w + 1} \sum_{j=-k_w}^{k_w} H(x + j). \tag{6.17}$$

The navigation reference is extracted as the column index corresponding to the profile maximum:

$$x_{\text{target}} = \arg\max_x \tilde{H}(x). \tag{6.18}$$

This formulation yields a single horizontal setpoint x_{target} rather than a continuous line model $x(y) = my + q$. As a result, it does not estimate heading error e_θ directly from the visual observation, and its output is sensitive to the total area of the segmented corridor within the field of view.

6.9.3 RANSAC-based contour baseline

The RANSAC-based contour baseline (`pf_ransac.py`) reproduces the classical boundary-fitting methodology prevalent in agricultural vision navigation [1, 14]. Unlike Method 4 and the center-point tracker, it operates directly over the segmented corridor mask without horizontal strip discretization or bottom-up association logic.

Following lower ROI cropping ($y_0 = \lfloor 0.30H \rfloor$) and morphological filtering, the algorithm extracts lateral boundary coordinates across all scanlines containing traversable path pixels. For each image row $y \in [y_0, H-1]$, the leftmost and rightmost corridor pixels define boundary points $(y, x_L(y))$ and $(y, x_R(y))$, provided the local corridor width $w(y) = x_R(y) - x_L(y) + 1$ satisfies $w(y) \geq 10$ px. This extracts two continuous point clouds, $\mathcal{P}_L$ and $\mathcal{P}_R$, containing all lateral boundary pixels along the corridor.

Boundary lines are estimated independently on $\mathcal{P}_L$ and $\mathcal{P}_R$ using Random Sample Consensus (RANSAC) [15]:

1. Across $T_{\max} = 100$ trials, two points are sampled uniformly at random from the boundary point cloud.
2. A candidate linear model $x(y) = m^{(t)}y + q^{(t)}$ is determined from the sampled pair.
3. Inliers are identified as points satisfying $|x_k - (m^{(t)}y_k + q^{(t)})| \leq \tau_{\text{res}}$, with residual threshold $\tau_{\text{res}} = 18.0$ px.
4. The model maximizing inlier cardinality is selected, followed by an ordinary least-squares refit over the complete inlier set.

The guidance centerline is calculated as the unweighted average of the left and right boundary models:

$$m_C = \frac{m_L + m_R}{2}, \qquad q_C = \frac{q_L + q_R}{2}. \tag{6.19}$$

By fitting dense boundary clouds directly, this approach avoids heuristic strip-tuning parameters. However, evaluating candidate models across hundreds of boundary points increases computational overhead, and stochastic sampling produces non-zero angular jitter across repeated runs on identical frames.

6.10 Comparative trade-offs and structural limitations

The four geometric extraction algorithms present distinct engineering trade-offs. No single method is optimal across all operational metrics. Table 6.3 summarizes

the structural characteristics, computational properties, and operational failure modes of each extractor.

Table 6.3: Structural comparison of the four geometric extraction algorithms.

Characteristic	`pf_histogram`	`pf_centri`	`pf_ransac`	Method 4
Output reference	Lateral column x_{target}	Line (m_C, q_C)	Line (m_C, q_C)	Line (m_C, q_C)
Orientation (e_θ) estimate	No	Yes	Yes	Yes
Deterministic execution	Yes ($\sigma_\omega = 0$)	Yes ($\sigma_\omega = 0$)	No ($\sigma_\omega > 0$)	Yes ($\sigma_\omega = 0$)
Computational complexity	$O(W \cdot H)$	$O(N)$	$O(T_{\max} \cdot \lvert\mathcal{P}\rvert)$	$O(N)$
Edge decoupling	N/A	No	Yes	Yes
Sensitivity to asymmetric noise	High	High	Low	Low
Bottom-up seed dependence	None	High	None	High
Local jump threshold tuning	None	Required	None	Required
Curvature representation	Poor	Moderate	Poor	Poor

6.10.1 Structural limitations of Method 4

While Method 4 decouples boundary estimation and avoids the stochastic jitter of RANSAC, it introduces specific structural constraints:

1. **Bottom-up seed dependence:** The tracking process initializes from candidate intervals detected within the bottom six strips ($y \in [H-1, H-6h_s]$). If the foreground mask immediately ahead of the vehicle contains heavy occlusions, soil debris, or severe false-negative patches, the extractor fails to identify a valid initial seed. Furthermore, if the algorithm locks onto a false boundary candidate at the base of the image, the spatial continuity constraint propagates this error upward through subsequent strips. In contrast, `pf_histogram` and `pf_ransac` aggregate pixels across the entire field of view simultaneously and do not depend on bottom-up temporal or spatial seeding.

2. **Sensitivity to tracking hyperparameters:** The spatial continuity gate $\Delta x_{\max} = 28$ px and the maximum allowable gap of three consecutive empty strips are tuned for a specific image resolution (224×224) and camera mounting angle. Pitch and roll disturbances induced by uneven terrain cause rapid shifts of the corridor in image space. When these shifts exceed the spatial gate between consecutive strips, tracking terminates prematurely, resulting in frame rejection.

3. **Linear corridor model constraint:** Method 4 models both boundaries as straight lines ($x = my + q$). In curved crop rows, contour-following trajectories, or headland turning areas, the linear model introduces systematic approximation error. While algorithms like the multi-ROI formulation of Li et al. [6] adopt cubic B-splines, Method 4 trades curve-fitting capacity for numerical stability and determinism on straight corridors.

4. **Vulnerability to extended unilateral gaps:** When crop gaps along one row exceed the three-strip threshold ($3 \times 6 = 18$ px in image space), the corresponding boundary tracker terminates. If only one boundary converges while the other is rejected, Method 4 rejects the frame unless an asymmetric single-edge fallback mode is explicitly enabled. Under identical conditions, the global histogram in `pf_histogram` continues to output a lateral setpoint, albeit biased toward the denser vegetation side.

Chapter 7

Robot Platform and Integration

This chapter describes the hardware and software components of the robot platform used for field validation. It covers the camera configuration, computing hardware, software architecture using ROS, and the deployment of the trained segmentation model for real-time inference.

7.1 Robot hardware and sensors

7.1.1 Robot platform

The experimental platform is a custom-built mobile robot prototype, named Demetra, developed for agricultural inspection and autonomous navigation in row-crop environments. The chassis features a four-wheel steering (4WS) and four-wheel drive (4WD) kinematic configuration. All four wheels are independently driven, and both the front and rear axles provide active steering.

Steering operates in a symmetric contra-steering mode (opposite-phase steering), where the front wheels steer by an angle $\delta_f = \delta$ and the rear wheels steer by an equal and opposite angle $\delta_r = -\delta$. This configuration constrains the instantaneous center of rotation (ICR) to lie on the transverse axis passing through the geometric center of the chassis ($x = 0$). Relative to a conventional front-wheel Ackermann steering model with wheelbase L, symmetric four-wheel contra-steering halves the turning radius from $R = L/\tan\delta$ to:

$$R = \frac{L}{2\tan\delta}. \tag{7.1}$$

This reduction in turning radius enables tight turns at row headlands without requiring the robot to swing wide into adjacent crop beds. In addition, the vehicle

sideslip angle at the center of geometry remains zero under ideal kinematic rolling conditions ($\beta = 0$), so the heading of the chassis aligns with the tangent to the path.

▶ DA COMPLETARE: Dimensioni fisiche e peso del telaio

Inserire i valori geometrici misurati sul prototipo reale:

- Passo L (distanza tra asse anteriore e posteriore in metri) e carreggiata T (distanza trasversale tra le ruote in metri).
- Raggio nominale della ruota r_w e altezza libera da terra del telaio (ground clearance).
- Massa totale del veicolo (kg) comprensiva di pacco batterie, sensori e payload.

The traction subsystem comprises four DJI RoboMaster M3508 brushless DC motors, each coupled to an integrated 19:1 planetary gearbox. Operating at a nominal voltage of 24 V, each motor unit delivers up to 3.0 N·m of continuous torque with a no-load output shaft speed of 482 rpm (peak torque up to 5.0 N·m). Speed regulation is handled by four individual DJI RoboMaster C620 electronic speed controllers (ESCs) implementing field-oriented control (FOC) over a CAN bus interface at 1 Mbps, with continuous current limits of 20 A per channel.

Active steering is actuated by four CubeMars AK70-10 KV100 robotic actuators. Each steering unit integrates a brushless DC motor, a 10:1 planetary gearhead, a 14-bit magnetic absolute encoder, and a motor driver in a single housing. A cross-roller bearing supports simultaneous axial, radial, and tilting loads transmitted from the ground through the wheel uprights. The actuators produce a continuous torque of 8.3 N·m and a peak torque of 24.8 N·m under 24 V operation. The integrated 14-bit encoder provides angular position feedback with a resolution of 0.022°, used for closed-loop position control over a secondary CAN bus network.

Low-level actuator control, safety monitoring, and sensor feedback are executed on a DJI RoboMaster Development Board Type C. The board houses an STM32F407IGH6 microcontroller (ARM Cortex-M4 core running at 168 MHz with hardware floating-point support). Two independent CAN bus controllers interface directly with the four traction ESCs and the four steering actuators. The board also incorporates a 6-axis inertial measurement unit (IMU) mounted on a mechanical vibration-damping structure. Deterministic firmware on the microcontroller executes position and velocity loops at fixed intervals, parses motion commands from the onboard computer, and transmits wheel encoder odometry and IMU telemetry.

7.1.2 Camera configuration

Visual sensing is provided by an Intel RealSense D435 camera mounted on the front section of the robot frame. The sensor connects to the onboard computer

[SPAZIO FIGURA / SCHEMA]

Physical mobile robot platform and front camera mounting setup

Fotografia della piattaforma mobile con indicazione della telecamera RGB frontale, dell'altezza di montaggio h dal terreno, dell'angolo di inclinazione (pitch) verso il suolo e dell'alloggiamento del calcolatore di bordo.

Figure 7.1: Physical mobile robot platform and front camera mounting setup

via a USB 3.2 Gen 1 interface. For the navigation pipeline, the camera operates in RGB streaming mode at a resolution of 640×480 pixels and a frame rate of 30 frames per second (fps). The stream is captured as JPEG-compressed frames to minimize transfer overhead across the ROS 2 transport layer. The camera is positioned at height h above the ground with a downward tilt angle α toward the inter-row path, providing a field of view that encompasses the corridor surface and adjacent crop boundaries in front of the vehicle.

7.1.3 Computing platform

Onboard processing is executed on an NVIDIA Jetson AGX Orin Developer Kit (64 GB version). The computing unit incorporates a 12-core ARM Cortex-A78AE v8.2 64-bit CPU operating at 2.20 GHz and an NVIDIA Ampere architecture GPU with 2048 CUDA cores and 64 Tensor cores. The module delivers up to 275 TOPS of sparse INT8 compute capability and 138 TFLOPS in FP16 Tensor Core operations.

The system shares 64 GB of unified LPDDR5 memory between the CPU and GPU over a 256-bit bus, providing a memory bandwidth of 204.8 GB/s. Persistent storage and dataset logging use a 512 GB NVMe SSD mounted over an M.2 PCIe Gen4 $\times 4$ interface at `/mnt/nvme`, leaving the 64 GB onboard eMMC memory dedicated to the base operating system.

The software stack runs on Ubuntu 22.04.5 LTS with Linux kernel `5.15.148-tegra` and NVIDIA Linux for Tegra (L4T) release R36.4.4 under JetPack 6.1. The complete robotics and deep learning environment is containerized using Docker and the NVIDIA Container Runtime, providing pass-through access to the GPU, hardware accelerators, and USB character devices. Within the container, ROS 2 Humble Hawksbill coordinates image acquisition, neural network inference with ONNX Runtime 1.23.0 using the `CUDAExecutionProvider`, geometric line extraction, and low-level serial communication.

7.2 Software architecture

7.2.1 ROS integration

The software pipeline is implemented using the Robot Operating System (ROS). ROS provides a modular architecture where perception, geometric extraction, and control components communicate through topics and services.

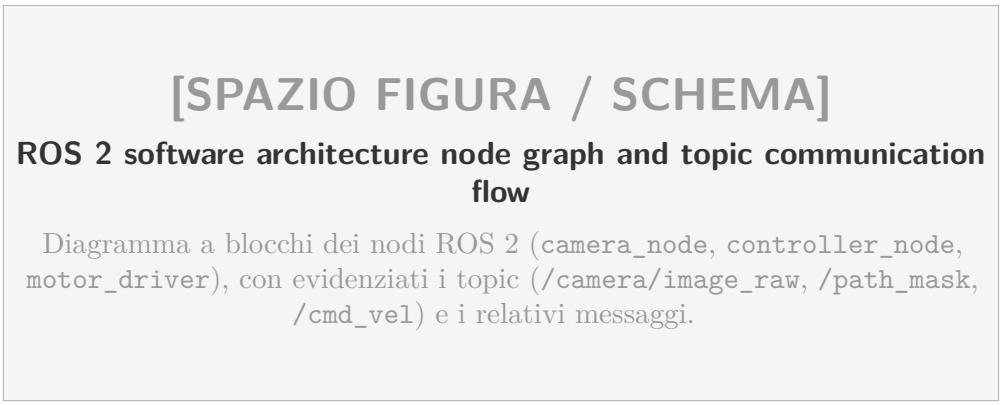


Figure 7.2: ROS 2 software architecture node graph and topic communication flow

▶ DA COMPLETARE: Architettura dei nodi ROS 2 e politiche QoS

Descrivere l'organizzazione software nel package ROS 2 `seg_controller_onnx`:

- Grafo dei nodi attivi e rispettivi ruoli nel ciclo di percezione-azione.
- Topic scambiati: ricezione stream telecamera compresso, pubblicazione della maschera del sentiero, coordinate della linea di navigazione e comandi motore `/cmd_vel`.
- Politiche di Quality of Service (QoS): adozione di code a profondità unitaria (`depth=1`) con policy `BEST_EFFORT` per evitare l'accumulo di latenza su frame arretrati.

Perception node

The perception node receives camera images, runs the segmentation model, and publishes the path mask. Input images are preprocessed according to the training configuration before inference.

▶ DA COMPLETARE: Dettagli implementativi del nodo di percezione

Descrivere le operazioni eseguite alla ricezione di ogni frame:

- Decodifica dell'immagine compressa JPEG tramite `cv2.imdecode`.
- Ridimensionamento e normalizzazione del tensore d'ingresso (media e deviazione standard di ImageNet).
- Invocazione dell'inferenza tramite la sessione ONNX Runtime con `CUDAExecutionProvider`.
- Estrazione della maschera binaria di navigazione tramite operatore `argmax`.

Geometric extraction node

The geometric extraction node subscribes to the segmentation mask, applies Method 3 or Method 4 from Chapter 6, and publishes the navigation reference as a line or position message.

▶ DA COMPLETARE: Integrazione dell'estratore geometrico e tracking temporale

Dettagliare l'elaborazione geometrica:

- Suddivisione della maschera in strisce orizzontali e selezione dei candidati.
- Applicazione del filtro particellare (`pf3.py` o `pf4.py`) per tracciare la linea centrale nel tempo, riducendo le oscillazioni introdotte da buchi nella vegetazione.
- Gestione dei frame non validi (reiezione in caso di confidenza o supporto insufficiente).

Control node and error conditioning

The control node converts the image-space geometric reference into linear and angular velocity commands (v, ω) published on the `/cmd_vel` topic. Tracking errors extracted from the segmentation mask at reference row $y_{\text{ref}} = 0.50 \cdot H$ ($y = 112$ px for a 224×224 input resolution) are subject to sensor noise and transient occlusions. To ensure smooth closed-loop actuation without erratic steering adjustments, raw lateral pixel errors e_{px} pass through a multi-stage conditioning filter before entering the steering law.

Dual-regime recursive smoothing and jump rejection. The core filter operates as an exponential moving average (EMA) conditioned on the path finder tracking state $\text{status}_{\text{pf}} \in \{\text{ok}, \text{degraded}\}$. The filter smoothing factor α and jump threshold J_{thr} switch dynamically between two operating regimes:

$$(\alpha, J_{\text{thr}}) = \begin{cases} (0.20,\ 40.0 \text{ px}) & \text{if } \text{status}_{\text{pf}} = \text{ok}, \\ (0.18,\ 38.0 \text{ px}) & \text{if } \text{status}_{\text{pf}} = \text{degraded}. \end{cases} \tag{7.2}$$

At each sampled frame k, the discrepancy between incoming measurement $e_{\text{px},k}$ and prior filtered state e_{k-1} is evaluated as $\Delta_k = |e_{\text{px},k} - e_{k-1}|$. When $\Delta_k \leq J_{\text{thr}}$, the measurement is accepted as nominal and updated via standard EMA:

$$e_k = (1 - \alpha) e_{k-1} + \alpha\, e_{\text{px},k}, \tag{7.3}$$

with the consecutive jump counter reset to zero ($c_{\text{jump}} = 0$).

Conversely, when $\Delta_k > J_{\text{thr}}$, the counter increments ($c_{\text{jump}} \leftarrow c_{\text{jump}} + 1$). If the anomaly occurs on a single isolated frame ($c_{\text{jump}} = 1$), it is treated as a spurious segmentation outlier and heavily attenuated:

$$e_k = 0.80\, e_{k-1} + 0.20\, e_{\text{px},k}. \tag{7.4}$$

If the large displacement persists across consecutive frames ($c_{\text{jump}} \geq 2$), the filter classifies the variation as a genuine path change, resets $c_{\text{jump}} = 0$, and assigns $e_k = e_{\text{px},k}$ directly to avoid phase lag.

Temporal hold and command deadband. When the path finder reports a lost tracking state or when no valid centerline is detected ($e_{\text{px}} = \text{None}$), the filter initiates a zero-order temporal hold:

$$e_k = e_{\text{last_valid}} \quad \text{for } \Delta t \leq t_{\text{hold}}, \tag{7.5}$$

where $t_{\text{hold}} = 0.50$ s (≈ 15 frames at 30 fps). If perception dropouts exceed 0.50 s, the filter clears its internal state and issues an emergency stop command ($v = 0$, $\omega = 0$).

To eliminate high-frequency actuator jitter when the vehicle is aligned with the corridor, an output deadband of ±4.0 px is applied:

$$e_{\mathrm{cmd},k} = \begin{cases} 0.0 & \text{if } |e_k| < 4.0 \text{ px}, \\ e_k & \text{otherwise.} \end{cases} \tag{7.6}$$

The deadband alters only command output $e_{\mathrm{cmd},k}$, preserving internal state e_k to avoid filter deadlock when crossing the zero-error threshold.

Velocity command generation. The filtered lateral error is normalized relative to the image half-width, $e_{\mathrm{norm}} = e_{\mathrm{cmd}}/(W/2)$, and scaled by the geometric extraction confidence $C \in [0, 1]$:

$$\omega = -K_{p,\mathrm{lat}} \cdot (0.6 + 0.4 \cdot C) \cdot e_{\mathrm{norm}}, \tag{7.7}$$

where $K_{p,\mathrm{lat}} = 1.2$ is the lateral proportional gain. Calculated angular velocity is clamped within $[-\omega_{\max}, \omega_{\max}]$, while forward linear speed is set to $v = v_{\mathrm{nom}}$. Table 7.1 summarizes the nominal parameters deployed on the robot platform.

Table 7.1: Nominal parameters of the error conditioning filter and steering controller deployed on Demetra.

Parameter	Symbol	Nominal value
Input resolution ($W \times H$)	–	224×224 px
Reference evaluation row	y_{ref}	112 px ($0.50 \cdot H$)
Smoothing factor (regime `ok`)	α_{ok}	0.20
Smoothing factor (regime `degraded`)	α_{deg}	0.18
Jump threshold (regime `ok`)	$J_{\mathrm{thr,ok}}$	40.0 px
Jump threshold (regime `degraded`)	$J_{\mathrm{thr,deg}}$	38.0 px
Outlier persistence window	N_{persist}	2 frames
Temporal hold duration	t_{hold}	0.50 s ($\approx$ 15 frames)
Output deadband	δ_{db}	±4.0 px
Lateral proportional gain	$K_{p,\mathrm{lat}}$	1.2
Maximum angular velocity	$\omega_{\max}$	0.58 rad/s
Nominal linear speed	v_{nom}	0.20 m/s

7.2.2 Perception pipeline implementation

▶ DA COMPLETARE: Struttura a callback sincrona e carico delle immagini di debug

Descrivere il funzionamento del nodo unificato in un unico thread:

- Sequenzialità del ciclo: decodifica JPEG → inferenza ONNX → particle filter → generazione comandi → compressione e pubblicazione di 3 immagini di diagnostica visiva.
- Analisi dell'impatto prestazionale: la compressione JPEG delle 3 immagini di debug rappresenta una frazione significativa del tempo totale di callback.

7.2.3 Control interface

The control node translates the guidance reference into velocity commands published as `geometry_msgs/msg/Twist` messages on the `/cmd_vel` topic. These commands specify the forward linear speed v along the longitudinal chassis axis and the angular velocity ω about the vertical yaw axis.

A dedicated serial communication node, `demetra_serial_driver`, interfaces the ROS 2 environment with the low-level STM32 microcontroller. The physical connection uses an FTDI USB-to-UART TTL converter operating at a baud rate of 115200 bps with an update loop running at 100 Hz (10 ms period). Each transmitted packet consists of 24 bytes containing six 32-bit floating-point values followed by a checksum for transmission validation.

On the microcontroller, inverse kinematics converts the target (v, ω) command into individual steering angles and wheel rotation speeds according to the symmetric 4WS model. For a vehicle with wheelbase L and track width T, the nominal steering angle δ is:

$$\delta = \arctan\left(\frac{L\omega}{2v}\right), \tag{7.8}$$

with the front axle steering at $+\delta$ and the rear axle steering at $-\delta$. The individual turn radius for the center of each axle is $R = L/(2\tan\delta)$. To account for track width T and reduce wheel scuffing during cornering, individual steering angles for the left and right wheels follow dual-Ackermann geometry:

$$\delta_L = \arctan\left(\frac{L/2}{R - T/2}\right), \qquad \delta_R = \arctan\left(\frac{L/2}{R + T/2}\right), \tag{7.9}$$

with $\delta_{FL} = \delta_L$, $\delta_{FR} = \delta_R$, $\delta_{RL} = -\delta_L$, and $\delta_{RR} = -\delta_R$. The corresponding wheel linear velocities v_i for each wheel $i \in \{FL, FR, RL, RR\}$ are:

$$v_i = \omega\sqrt{\left(\frac{L}{2}\right)^2 + R_i^2}, \tag{7.10}$$

where R_i is the lateral distance from the ICR to the wheel center plane. Target wheel angular velocities $\omega_{w,i} = v_i/r_w$ (with wheel radius r_w) are converted into motor speed setpoints for the C620 ESCs via the 19:1 gear ratio, while the steering angles are tracked by the AK70-10 actuators via the internal 14-bit encoder position loops.

7.3 Deployment configuration

7.3.1 Model export to ONNX

The trained LR-ASPP model is exported to ONNX format for deployment. ONNX Runtime provides optimized inference across different hardware platforms. The export procedure preserves model weights and input/output specifications from the training checkpoint.

▶ DA COMPLETARE: Procedura di esportazione ONNX

Documentare i comandi e le opzioni dello script di esportazione da PyTorch (.pt/.pth) a ONNX (.onnx):

- Funzione `torch.onnx.export` con parametri di input (batch size fisso a 1, risoluzione d'ingresso $H \times W$, Opset 13/14).
- Verifica di equivalenza numerica tra i logit prodotti da PyTorch e quelli generati dal modello esportato in ONNX.

7.3.2 Real-time inference

The deployed model processes camera frames at the rate required for closed-loop control. Input preprocessing matches the training transformations. The segmentation mask is decoded and passed to the geometric extraction stage.

▶ DA COMPLETARE: Configurazione del runtime ONNX

Specificare i dettagli dell'ambiente di esecuzione:

- Configurazione dell'allocazione di memoria e provider attivo (`CUDAExecutionProvider`).
- Tipologia di quantizzazione/precisione (es. FP32 standard vs FP16) e consumo di memoria VRAM sulla GPU Jetson.

7.3.3 Timing and latency

End-to-end latency includes image acquisition, preprocessing, segmentation inference, geometric extraction, and control computation. The total pipeline latency

must remain below the control update period to maintain stable navigation.

[SPAZIO FIGURA / SCHEMA]

Execution time breakdown of the synchronous perception-control cycle on NVIDIA Jetson

Grafico della scomposizione del tempo di ciclo medio su Jetson: decodifica JPEG (10–15 ms), inferenza ONNX su GPU (10–15 ms), filtro geometrico/particellare (5–10 ms), codifica delle 3 immagini JPEG di debug (15–25 ms). Tempo totale per ciclo: 42–76 ms (20 Hz).

Figure 7.3: Execution time breakdown of the synchronous perception-control cycle on NVIDIA Jetson

▶ DA COMPLETARE: Analisi delle latenze misurate e frequenza a 20 Hz (dati reali)

Riportare i risultati empirici ricavati dai test sul Jetson:

- Tempo totale di esecuzione compreso tra 42 e 76 ms per ciclo (media $\approx$ 50 ms), corrispondente a una frequenza di controllo di circa 20 Hz a fronte di una frequenza di cattura della telecamera di 30 fps.
- Analisi del frame dropping indotto dalla callback sincrona con `depth=1` e `BEST_EFFORT` (viene scartato circa un frame ogni tre per mantenere il controllo sul dato più recente).
- Strategie per raggiungere i 30 Hz: (1) pubblicare le immagini di debug a frequenza ridotta o in un thread dedicato; (2) ottimizzare il modello tramite NVIDIA TensorRT; (3) disaccoppiare decodifica e inferenza con architettura multi-thread.

7.3.4 Software-in-the-Loop pre-validation

Before deploying the navigation pipeline on the physical robot, the stability of the full closed-loop chain (segmentation, geometric extraction, error conditioning, steering law, and vehicle kinematics) was verified in a lightweight Software-in-the-Loop (SITL) environment running on a desktop workstation.

SITL architecture

The SITL node loads a single real RGB frame acquired by the Intel RealSense D435 camera on an actual crop row. Rather than rendering a synthetic 3D scene, the simulator models the robot's lateral displacement and heading variation as a

horizontal translation of the camera viewport over the original image. The frame is padded with reflective borders (400 pixels per side) to allow displacements of up to ±0.45 m without introducing black edges.

The pixel shift Δx applied at each simulation step combines two terms derived from the D435 optical parameters:

$$\Delta x = -y \cdot \frac{f_x}{D} + \theta \cdot f_x, \tag{7.11}$$

where y is the lateral offset of the robot from the corridor center (in meters), θ is the heading angle relative to the row axis (in radians), $f_x = 465$ px is the horizontal focal length of the D435 at 640×480 resolution, and $D = 1.25$ m is the lookahead distance corresponding to the reference evaluation row ($y_{\text{ref}} = 0.50 \cdot H$).

Kinematic model

The SITL integrates the same 4WS symmetric contra-steering kinematics implemented in the onboard firmware (`kinematics.cpp`), with identical parameters: wheelbase $L = 0.40$ m, wheel diameter $d_w = 0.192$ m, and mechanical steering saturation at ±30.0°. At each time step $\Delta t = 1/30$ s, the node receives (v, ω) from the `/cmd_vel` topic published by the real `controller_node`, computes the effective steering angle:

$$\delta = \arctan\!\left(\frac{L \cdot \omega}{2\,v}\right), \qquad \omega_{\text{eff}} = \frac{2\,v\,\tan\delta}{L}, \tag{7.12}$$

and updates the planar state:

$$\theta_{k+1} = \theta_k + \omega_{\text{eff}} \cdot \Delta t, \tag{7.13}$$

$$y_{k+1} = y_k - v \sin\theta_k \cdot \Delta t. \tag{7.14}$$

The translated frame is then published on the same ROS 2 topic used by the physical camera (`/camera/camera/color/image_raw/compressed`), so the `controller_node` processes it without any code modification.

Interactive perturbation testing

Keyboard inputs allow the operator to inject controlled lateral perturbations (±4 cm or ±10 cm) during execution, simulating sudden displacement events such as wheel slip on soft soil or an external push. This procedure was used to verify that the error conditioning filter and steering controller recover to the corridor center within approximately 10–15 frames ($\sim$ 0.5 s at 30 fps), without residual oscillations and without saturating the ±30° mechanical steering limit. Table 7.1 lists the filter and control parameters validated through these SITL tests before field deployment.

Chapter 8

Experimental Validation

This chapter reports the experimental evaluation of the segmentation models, geometric extractors, and integrated navigation system. It covers offline evaluation using recorded images and, if completed, closed-loop field trials.

8.1 Offline evaluation protocol

8.1.1 Segmentation metrics

Segmentation performance is measured using intersection over union (IoU) for each semantic class. For a class c, IoU is defined as:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{8.1}$$

where true positives, false positives, and false negatives are computed against the manual reference masks.

Mean IoU (mIoU) averages the per-class IoU values. For the four-class configuration, mIoU includes background, plants, path, and covering material. Path IoU is the task-specific metric most relevant for navigation.

Architecture comparison results

The architecture comparison results reported in Section 5.3.3 demonstrate that LR-ASPP achieves path IoU of 0.9546 with inference time of 10.41 ms, compared to FCN (0.9621, 58.24 ms) and DeepLabV3 (0.9651, 86.87 ms). The 1.2 percentage point mIoU deficit is offset by a 5–8× reduction in inference latency.

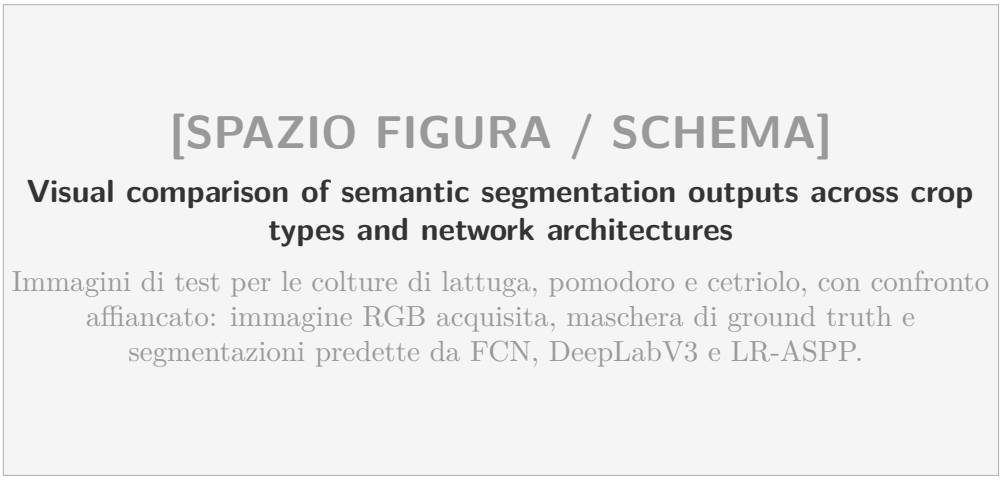


Figure 8.1: Visual comparison of semantic segmentation outputs across crop types and network architectures

Supervision strategy results

▶ DA COMPLETARE: Risultati comparativi delle strategie di supervisione

Inserire la tabella comparativa e commentare i risultati delle strategie di annotazione sul test set:

- Confronto tra modello addestrato su maschere manuali a 4 classi vs modello addestrato su maschere generate da SAM 3 a 4 classi.
- Valutazione della configurazione a 3 classi (piante, sentiero, sfondo) e della configurazione binaria (solo sentiero).
- Analisi delle prestazioni sia sull'intero test set che sul sottoinsieme operativo di navigazione.
- Discussione dell'efficacia di SAM 3 nel ridurre il tempo di annotazione rispetto alla qualità finale della maschera.

8.1.2 Geometric reference accuracy

Geometric reference quality is assessed by evaluating four extraction strategies on identical segmentation masks: the proposed dual-edge extractor with continuity tracking (Method 4), the RANSAC-based contour baseline (`pf_ransac.py`), the center-point strip tracking baseline (`pf_centri.py`), and the column-histogram baseline (`pf_histogram.py`). Performance metrics comprise:

- Reference line parameters: estimated slope m_C and lateral intercept q_C (for linear models);
- Lateral deviation e_y and heading orientation error e_θ evaluated against manual centerline annotations;
- Sensitivity to asymmetric disturbances, quantified by the lateral deviation induced by unilateral crop intrusions or shadows;
- Rejection rate: percentage of test images where extraction fails validation checks (status classified as lost);
- Execution time per frame measured on the onboard NVIDIA Jetson Orin Nano platform.

Quantitative comparison of geometric extractors

Table 8.1 reports offline performance metrics for the four geometric extraction algorithms evaluated on recorded test sequences. Each algorithm processes identical segmentation masks from the lettuce, tomato, and cucumber datasets.

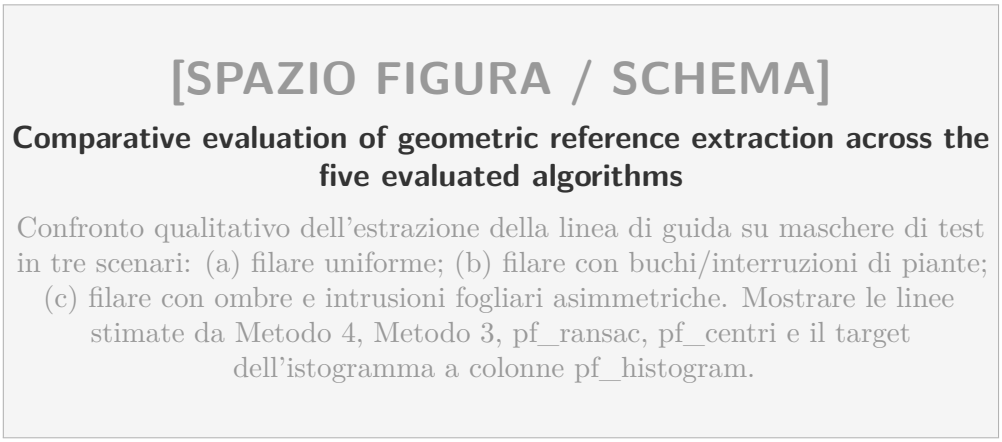


Figure 8.2: Comparative evaluation of geometric reference extraction across the five evaluated algorithms

Table 8.1: Offline benchmark of geometric extraction algorithms on multi-crop test sequences.

Metric	`pf_histogram`	`pf_centri`	`pf_ransac`	Method 4
Lateral RMSE (cm)	—	—	—	—
Angular RMSE (deg)	—	—	—	—
Angular jitter σ_ω (deg)	0.00	0.00	—	0.00
Rejection rate (%)	—	—	—	—
Execution time (ms)	—	—	—	—
Deterministic	Yes	Yes	No	Yes

Angular jitter σ_ω measures the standard deviation of the estimated centerline slope across 100 repeated executions on the same input frame. Deterministic algorithms produce $\sigma_\omega = 0.00$ by definition, while `pf_ransac` exhibits non-zero jitter due to its stochastic sampling strategy.

Asymmetric disturbance sensitivity

Asymmetric corridor disturbances—such as protruding leaves, localized weeds, or shadows extending along one side—test the ability of each algorithm to maintain corridor centering when the segmentation mask exhibits unilateral bias. Algorithms that collapse bilateral boundaries into a single centerline estimate before fitting (such as `pf_centri`) shift laterally toward the intrusion. Method 4 and `pf_ransac` isolate each boundary and compute the centerline analytically, reducing sensitivity to unilateral noise.

Failure mode analysis

Extraction failures occur when validation checks reject the computed reference as unreliable. Common failure modes include:

- **Tracking loss:** Insufficient mask continuity across horizontal strips prevents bottom-up tracking initialization.
- **Angular drift from corrupted centroids:** Single outlier intervals dominate the centroid-based fit in `pf_centri`, producing reference lines with excessive slope.
- **Histogram target jumps:** The column-histogram minimum in `pf_histogram` shifts abruptly between frames when corridor width varies or when noise perturbs column sums.
- **RANSAC convergence variability:** In sparse or highly noisy masks, `pf_ransac` may fail to identify a consistent inlier set within the iteration budget, leading to sporadic rejections.

8.2 Field validation

Field validation evaluates the integrated navigation system on the physical robot Demetra operating autonomously inside crop rows. Unlike offline evaluation, which isolates the geometric extractor on recorded images, closed-loop field tests exercise the full chain from camera acquisition through segmentation, geometric fitting, error conditioning, and steering actuation in the presence of real-world disturbances.

8.2.1 Ground-truth reference: laser sensor

To assess the robot's physical centering independently of the onboard vision pipeline under test, closed-loop field validation integrates an auxiliary laser sensor.

▶ DA COMPLETARE: Dettagli del sensore laser utilizzato per i test in campo

Inserire le specifiche esatte del sensore laser impiegato nei test:

- Modello, costruttore e tipologia di misura (ad es. sensore di distanza laser, barriera ottica o profilometro).
- Configurazione di montaggio sul robot Demetra (posizione e orientamento).
- Interfaccia di comunicazione e frequenza di campionamento.

The laser measurement provides an independent reference signal to monitor the robot's lateral alignment along the crop row and evaluate whether the trajectory remains within the allowable tolerance corridor.

8.2.2 Field evaluation metrics

The following metrics are computed for each test run. All algorithms under comparison (Method 4, `pf_centri`, `pf_ransac`, and `pf_histogram`) are evaluated on the same physical row under equivalent conditions.

Laser centering rate

The laser centering rate P_{laser} quantifies the fraction of the autonomous run during which the robot remains centered within the laser reference corridor:

$$P_{\text{laser}} = \frac{T_{\text{on}}}{T_{\text{total}}} \times 100\ [\%], \tag{8.2}$$

where T_{on} is the cumulative time with positive laser detection and T_{total} is the total duration of the autonomous run. A higher P_{laser} indicates that the guidance algorithm maintains the robot closer to the physical corridor center for a larger fraction of the mission.

Number of centering losses

The number of centering loss events N_{exit} counts how many times the laser detector transitions from active (1) to inactive (0) during a run. Each event corresponds to the robot drifting outside the acceptable centering band. Frequent transitions indicate oscillatory tracking behavior, even if the overall centering rate P_{laser} remains acceptable.

Mission completion rate

The mission completion rate S_{mission} measures the fraction of the test row traversed autonomously without requiring manual intervention or emergency stop:

$$S_{\text{mission}} = \frac{L_{\text{auto}}}{L_{\text{row}}} \times 100 \ [\%], \tag{8.3}$$

where L_{row} is the total length of the test row and L_{auto} is the distance covered before any intervention. A run terminated by collision with crop plants or by operator emergency stop records $S_{\text{mission}} < 100\%$.

Steering stability

Steering stability is evaluated from the angular velocity commands ω_k recorded on the `/cmd_vel` topic. Two indicators are computed:

- The standard deviation of angular velocity, $\sigma_\omega = \text{std}(\omega_k)$, which quantifies overall steering activity;
- The mean absolute jitter, $J_\omega = \frac{1}{N-1} \sum_{k=1}^{N-1} |\omega_{k+1} - \omega_k|$, which isolates high-frequency direction reversals independent of steady-state turning.

Lower values of both indicators correspond to smoother robot motion and reduced mechanical wear on the CubeMars AK70-10 steering actuators.

Effective forward speed

The effective forward speed v_{eff} is the ratio between the distance traversed and the elapsed time. If the error conditioning filter enters the temporal hold state and issues stop commands ($v = 0$), the effective speed drops below the nominal setpoint $v_{\text{nom}} = 0.20$ m/s. This metric captures how frequently each algorithm loses the guidance reference and forces the robot to halt.

8.2.3 Test conditions

▶ DA COMPLETARE: Descrizione delle condizioni sperimentali in campo

Dettagliare lo scenario delle prove sperimentali:

- Località del test, tipologia di coltura, sesto d'impianto (distanza tra i filari e altezza delle piante).
- Lunghezza del percorso navigato, tipologia del fondo (terreno lavorato, teli plastici pacciamanti, presenza di malerbe).
- Configurazione del sensore laser di riferimento (modello, posizionamento sul robot o nel filare, principio di misura).
- Condizioni meteorologiche e di luce (sole diretto, cielo coperto, ombre proiettate dalla vegetazione) e numero di ripetizioni effettuate per ciascun algoritmo.

8.2.4 Closed-loop navigation results

[SPAZIO FIGURA / SCHEMA]

Lateral tracking error and heading error profiles during inter-row autonomous navigation

Grafico dell'errore di inseguimento laterale e_y (cm) e dell'errore di orientamento e_θ (gradi) registrati nel tempo durante la navigazione del robot mobile all'interno dei filari coltivati.

Figure 8.3: Lateral tracking error and heading error profiles during inter-row autonomous navigation

▶ DA COMPLETARE: Risultati quantitativi di navigazione ad anello chiuso

Riportare i dati raccolti durante le prove in campo con il robot Demetra:

- Tabella comparativa tra i 4 algoritmi (Metodo 4, pf_ransac, pf_centri, pf_histogram) con le metriche definite in Sezione 8.2.2: P_{laser}, N_{exit}, S_{mission}, σ_ω, J_ω e v_{eff}.
- Grafico temporale del segnale del sensore laser sovrapposto alla velocità angolare comandata $\omega(t)$ per ciascun algoritmo.
- Analisi degli eventi critici: corrispondenza tra le anomalie del segnale laser e i frame classificati come `lost` dal path finder.

8.2.5 Qualitative observations

▶ DA COMPLETARE: Osservazioni qualitative sul comportamento del sistema

Discutere gli aspetti qualitativi emersi durante i test:

- Differenze visibili nella fluidità di guida tra Metodo 4 e pf_centri (oscillazioni laterali, scatti di sterzo).
- Comportamento in presenza di interruzioni (gaps) nella fila di piante e al variare dell'illuminazione.
- Casi limite riscontrati (es. testate del campo, biforcazioni del corridoio, erbacce invadenti).

8.3 Discussion

8.3.1 Segmentation performance

The architecture comparison demonstrates that LR-ASPP provides a practical accuracy–speed trade-off for robotic deployment. Its path IoU of 0.9546 on manually annotated lettuce is sufficient for geometric extraction, while its inference time supports real-time operation.

The supervision strategy comparison addresses the effort required for dataset preparation. SAM 3-assisted annotation reduces manual labeling time, but its quality and the need for inspection remain to be quantified systematically.

8.3.2 Geometric extraction trade-offs

The comparative evaluation highlights structural trade-offs across the four geometric extraction paradigms. No single algorithm demonstrates unconditional superiority across all operational criteria:

- **Method 4 versus `pf_ransac`:** Method 4 achieves strict determinism ($\sigma_\omega = 0.00°$) and executes in sub-millisecond time (< 0.1 ms), avoiding the iterative hypothesis-generation overhead and stochastic jitter characteristic of RANSAC. However, `pf_ransac` operates on unstructured point clouds without requiring horizontal discretization or sequential gating. When canopy boundaries contain extended irregular gaps, RANSAC can sample valid inliers across non-adjacent rows, whereas Method 4 terminates tracking when empty strips exceed the continuity threshold ($k_{\text{gap}} > 3$).
- **Method 4 versus `pf_centri`:** By maintaining separate linear models for the left and right corridor boundaries, Method 4 isolates asymmetric disturbances such as localized weed patches or protruding leaves. In contrast,

`pf_centri` collapses interval endpoints into midpoints prior to fitting, causing unilateral border noise to directly perturb the estimated centerline. Nevertheless, `pf_centri` requires fewer validation checks and operates with a simpler association logic than the decoupled dual-edge architecture.

- **Method 4 versus `pf_histogram`:** The column-histogram baseline (`pf_histogram`) is computationally the simplest approach, computing column projections in $O(W \cdot H)$ without strip partitioning, boundary association, or matrix inversion. It does not suffer from tracking divergence or initialization failure. Its primary drawback is geometric incompleteness: it produces only a horizontal setpoint x_{target} without estimating heading error e_θ. Furthermore, its target shifts when vegetation area varies unevenly along the corridor. Method 4 delivers a complete linear model (m_C, q_C) supporting full pose feedback, but incurs higher algorithmic complexity and depends on bottom-up mask continuity.

8.3.3 Limitations

The experimental findings and the proposed methodology are subject to several specific limitations:

- **Seed dependency in Method 4:** The bottom-up tracking logic relies on detecting a valid corridor interval within the bottom six strips of the region of interest. If the mask immediately in front of the vehicle is corrupted by motion blur, ground glare, or chassis shadows, tracking fails to initialize, causing frame rejection even when the upper corridor is clearly segmented. Unlike global projection methods, Method 4 lacks a mechanism to re-anchor tracking from the top down.

- **Fixed spatial gate calibration:** The continuity threshold $\Delta x_{\max} = 28$ px and strip discretization parameters are fixed in image coordinates. Significant vehicle pitch or roll on rough terrain causes rapid lateral shifts of the corridor boundaries between consecutive frames. When these shifts exceed the gate, the tracker rejects valid boundaries.

- **Straight-line corridor assumption:** All linear models evaluated (Method 4, `pf_ransac`, and `pf_centri`) assume locally straight, parallel corridor boundaries. They cannot follow curved planting geometries or guide the robot through headland turnaround maneuvers where corridor boundaries terminate or diverge.

- **Segmentation evaluation scope:** Ground-truth segmentation evaluation is conducted on manually annotated lettuce sequences. While the multi-crop

model is trained on tomato and cucumber, its generalization is evaluated through qualitative inspection and navigation performance rather than dense pixel-level error metrics on those crops.

- **Offline versus closed-loop dynamics:** Offline benchmarks quantify reference line accuracy on static recorded frames. They do not account for dynamic wheel-ground interaction, actuator saturation, or latency jitter during physical vehicle operation.

Chapter 9

Conclusions and Future Work

This chapter summarizes the main findings, discusses the contributions and limitations of the work, and identifies directions for future research.

9.1 Main contributions

This thesis investigated vision-based local guidance for agricultural row navigation through segmentation model selection, dataset preparation with model-assisted annotation, and geometric reference extraction. The main contributions are:

- **Architecture comparison for robotic segmentation:** FCN, DeepLabV3, and LR-ASPP were compared on manually annotated lettuce images. LR-ASPP was selected for its practical accuracy–speed trade-off, achieving path IoU of 0.9546 with 10.41 ms inference time compared to 58.24 ms and 86.87 ms for the other architectures. This 5–8× latency reduction supports real-time operation despite a 1.2 percentage point mIoU deficit.

- **SAM 3-assisted dataset preparation:** Multi-crop training datasets for lettuce, tomato, and cucumber were prepared using SAM 3 text-prompted segmentation. This reduced manual annotation effort, although systematic measurements of time savings and inspection requirements remain to be completed.

- **Evaluation of output label configurations:** Models with four-class, three-class, and binary path outputs were compared. The binary configuration aligns directly with the navigation task and was selected for robotic integration.

- **Edge-based geometric extractors:** Two methods for converting corridor masks into image-space navigation references were implemented. Both use horizontal strip extraction and independent fitting of left and right boundaries, differing in seed selection, association strategy, and validation checks.

9.2 Conclusions on segmentation and annotation

The architecture comparison establishes that LR-ASPP is suitable for real-time agricultural navigation when inference speed is a constraint. Its segmentation accuracy on lettuce is sufficient for path extraction, and its reduced computational cost makes it deployable on robot hardware.

SAM 3-assisted annotation demonstrates that foundation models can reduce manual labeling effort for agricultural datasets. The quality of automatically generated masks and their effect on trained model performance depend on crop appearance, text prompt formulation, and the need for manual inspection. Future work should measure annotation time systematically, including review and correction, to quantify the actual savings.

The choice of output label configuration affects both annotation effort and downstream geometric extraction. Binary path segmentation focuses the model directly on the navigation target, simplifying both the labeling task and the geometric stage. This comes at the cost of discarding plant and covering-material information that might be useful for other tasks.

9.3 Conclusions on geometric extraction

The geometric extraction stage converts binary corridor segmentation masks into navigation setpoints. The comparative study examined four distinct architectural paradigms:

1. Global column projection (`pf_histogram`) maximizes simplicity and eliminates tracking failure modes, but discards heading angle information and exhibits sensitivity to asymmetric vegetation distribution.

2. Centerline strip regression (`pf_centri`) provides both lateral offset and heading estimates with low computational demand, but transfers unilateral canopy noise directly into the navigation line.

3. Random sample consensus (`pf_ransac`) handles scattered boundary points and wide gaps without strip discretization, but introduces frame-to-frame stochastic jitter ($\sigma_\omega > 0$) and variable runtime.

4. Decoupled dual-edge tracking (Method 4) provides deterministic execution ($\sigma_\omega = 0.00°$), sub-millisecond latency, and boundary-level noise decoupling, but introduces strict dependencies on near-field seed detection and heuristic jump gates.

These characteristics demonstrate that no individual extractor is universally superior; selection depends on sensor setup, platform compute limits, and field layout.

▶ DA COMPLETARE: Sintesi quantitativa finale dell'estrazione geometrica e navigazione

Sintetizzare i dati conclusivi:

- Margine di precisione medio dell'asse stimato (scostamento in pixel e in centimetri).
- Tasso medio di affidabilità e di reiezione dei frame lungo i test eseguiti.
- Conferma della sostenibilità del controllo a 20 Hz su piattaforma embedded Jetson in condizioni di campo.

9.4 Limitations

The methodology and experimental findings are subject to several specific limitations:

- **Bottom-up seed dependency in Method 4:** The tracking algorithm initializes exclusively from the bottom six horizontal strips. Occlusions immediately in front of the camera (such as robot chassis shadows, soil buildup, or puddles) prevent track initialization and lead to frame rejections, even when the distant corridor is visible.

- **Geometric model constraints:** Method 4, `pf_ransac`, and `pf_centri` assume locally straight, parallel corridor boundaries. They cannot follow curved crop rows or navigate headland turnaround maneuvers where crop boundaries terminate or diverge.

- **Unilateral row dropout vulnerability:** If crop continuity along one row is broken by gaps exceeding three strips (18 px), Method 4 drops that boundary. Without an asymmetric single-edge fallback mode, this results in reference rejection, whereas global histogram projection continues to provide a lateral setpoint.

- **Image-space gate tuning:** The continuity gate ($\Delta x_{\max} = 28$ px) is defined in image coordinates. Significant vehicle pitch and roll on uneven soil cause

sudden lateral displacements of the corridor between consecutive scanlines, which can trigger premature tracking loss.

- **Dataset annotation coverage:** Segmentation models were quantitatively evaluated against dense manual ground truth on lettuce. Multi-crop training includes tomato and cucumber, but their segmentation accuracy was not verified against dense manual annotations for those specific crops.
- **Annotation effort measurement:** Time savings from SAM 3-assisted annotation were evaluated qualitatively. The comparison does not report logged inspection hours or the frequency of manual polygon corrections.
- **Closed-loop dynamics:** Offline benchmarks evaluate reference accuracy on static test images. They do not account for vehicle inertia, steering backlash, or dynamic latency variations under closed-loop field control.

9.5 Future work

Several directions for future research emerge from this work:

9.5.1 Dataset and annotation

- Measure annotation time systematically for manual and SAM 3-assisted workflows, including inspection and correction.
- Prepare manual ground-truth masks for tomato and cucumber to validate multi-crop model performance directly.
- Investigate active learning strategies to select images for manual annotation that improve model generalization most effectively.
- Explore other foundation models or fine-tuned SAM variants specialized for agricultural segmentation.

9.5.2 Segmentation and model deployment

- Evaluate additional lightweight architectures (e.g., BiSeNet, FastSCNN) for further latency reduction.
- Investigate quantization and model compression techniques to enable deployment on edge devices.
- Test temporal consistency methods that use previous frames to improve segmentation and reduce flickering.

- Explore uncertainty estimation to detect when segmentation confidence is low and fallback strategies are needed.

9.5.3 Geometric extraction

- Extend the geometric representation from linear models to piecewise linear or polynomial formulations to support curved rows and headland turns.
- Implement top-down recovery logic and single-edge asymmetric tracking to maintain guidance when bottom-up seeding fails or unilateral row dropouts occur.
- Adapt spatial gate thresholds dynamically using vehicle attitude telemetry from an onboard IMU to prevent false tracking loss during pitch and roll variations.
- Incorporate temporal Bayesian tracking (such as an Extended Kalman Filter) to smooth reference parameters across frame dropouts without incurring the stochastic jitter of RANSAC.

9.5.4 System integration and field validation

- Complete closed-loop field trials to validate end-to-end navigation performance under variable lighting, growth stages, and row conditions.
- Measure lateral tracking error and success rate over extended distances and multiple crop types.
- Integrate temporal filtering and prediction to handle brief reference loss or occlusions.
- Investigate fusion with other sensors (e.g., depth, IMU) to improve robustness.
- Develop a complete autonomy system including global path planning, obstacle avoidance, and task-specific behaviors.

9.6 Closing remarks

This thesis addressed the segmentation, dataset preparation, and geometric extraction stages of vision-based agricultural navigation. The selection of LR-ASPP balances accuracy and speed for robotic deployment. SAM 3-assisted annotation reduces manual effort, and the geometric extractors convert masks into navigation references using horizontal strip selection and independent edge fitting.

The work establishes a foundation for camera-based local guidance in structured crop environments. Future research should complete field validation, extend the geometric model to handle curves and gaps, and systematically measure annotation savings to guide dataset preparation strategies.

Bibliography

[1] Yuhao Bai, Baohua Zhang, Naimin Xu, Jun Zhou, Jiayou Shi, and Zhihua Diao. «Vision-based navigation and guidance for agricultural autonomous vehicles and robots: A review». In: *Computers and Electronics in Agriculture* 205, 107584 (Feb. 2023). DOI: 10.1016/j.compag.2022.107584 (cit. on pp. 1, 2, 4, 5, 9, 13, 53).

[2] Rajitha de Silva, Grzegorz Cielniak, Gang Wang, and Junfeng Gao. «Deep Learning-Based Crop Row Detection for Infield Navigation of Agri-Robots». In: *Journal of Field Robotics* 41.7 (2024), pp. 2299–2321. DOI: 10.1002/rob.22238 (cit. on pp. 1, 4, 7–10, 13, 15, 16).

[3] Alessandro Navone, Mauro Martini, Marco Ambrosio, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «GPS-Free Autonomous Navigation in Cluttered Tree Rows with Deep Semantic Segmentation». In: *Robotics and Autonomous Systems* 183, 104854 (Jan. 2025). DOI: 10.1016/j.robot.2024.104854 (cit. on pp. 1, 2, 4, 7, 8, 10–12, 15, 16, 52).

[4] Enrico Mendez, Javier Piña Camacho, Jesús Arturo Escobedo Cabello, and Alfonso Gómez-Espinosa. «Autonomous Navigation and Crop Row Detection in Vineyards Using Machine Vision with 2D Camera». In: *Automation* 4.4 (Sept. 2023), pp. 309–326. DOI: 10.3390/automation4040018 (cit. on pp. 2, 5, 7–9, 12, 15, 16).

[5] Johannes Kneip, Patrick Fleischmann, and Karsten Berns. «Crop Edge Detection Based on Stereo Vision». In: *Robotics and Autonomous Systems* 123, 103323 (Jan. 2020). DOI: 10.1016/j.robot.2019.103323 (cit. on pp. 4, 7, 12, 14–16).

[6] Xia Li, Junhao Su, Zhenchao Yue, and Fangtao Duan. «Adaptive Multi-ROI Agricultural Robot Navigation Line Extraction Based on Image Semantic Segmentation». In: *Sensors* 22.20, 7707 (Oct. 2022). DOI: 10.3390/s22207707 (cit. on pp. 5, 7, 10, 12, 14, 15, 17, 55).

[7] Jiqing Chen, Hu Qiang, Jiahua Wu, Guanwen Xu, and Zhikui Wang. «Navigation Path Extraction for Greenhouse Cucumber-Picking Robots Using the Prediction-Point Hough Transform». In: *Computers and Electronics in Agriculture* 180, 105911 (Jan. 2021). DOI: 10.1016/j.compag.2020.105911 (cit. on pp. 5, 7, 12, 15, 16).

[8] Alessandro Navone, Mauro Martini, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «Autonomous Navigation in Rows of Trees and High Crops with Deep Semantic Segmentation». In: *2023 European Conference on Mobile Robots (ECMR)*. IEEE, 2023, pp. 1–6. DOI: 10.1109/ECMR59166.2023.10256334 (cit. on p. 10).

[9] Jonathan Long, Evan Shelhamer, and Trevor Darrell. «Fully Convolutional Networks for Semantic Segmentation». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3431–3440. DOI: 10.1109/CVPR.2015.7298965 (cit. on pp. 10, 19).

[10] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. «Rethinking Atrous Convolution for Semantic Image Segmentation». In: *arXiv preprint arXiv:1706.05587* (2017) (cit. on pp. 10, 20, 23).

[11] Andrew Howard et al. «Searching for MobileNetV3». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 1314–1324. DOI: 10.1109/ICCV.2019.00140 (cit. on pp. 10, 23).

[12] Alexander Kirillov et al. «Segment Anything». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 4015–4026. arXiv: 2304.02643 [cs.CV] (cit. on pp. 11, 29, 30).

[13] Nicolas Carion et al. «SAM 3: Segment Anything with Concepts». In: *International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2511.16719 [cs.CV]. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/e0982cbc81401df3430ee1ff780dc7a2-Abstract-Conference.html (cit. on pp. 11, 29).

[14] Letian Wu, Yongzhi Cui, Huifeng Shi, Xiaoli Sun, Jiayan Yang, Xinwei Cao, Ping Zou, and Ya Liu. «An Improved DeepLabV3+-Based Method for Crop Row Segmentation and Navigation Line Extraction in Agricultural Fields». In: *Sensors* 26.10, 3142 (May 2026). DOI: 10.3390/s26103142 (cit. on pp. 13, 53).

[15] Martin A. Fischler and Robert C. Bolles. «Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography». In: *Communications of the ACM* 24.6 (June 1981), pp. 381–395. DOI: 10.1145/358669.358692 (cit. on pp. 13, 53).

[16] NumPy Developers. *numpy.polyfit: NumPy v2.4 Manual*. URL: `https://numpy.org/doc/2.4/reference/generated/numpy.polyfit.html` (visited on 09/13/2026) (cit. on p. 47).